

6. Semana 6. Cuantificación de incertidumbre: GUM y métodos Monte Carlo

Objetivo de la cátedra

Formular y propagar la incertidumbre de medición de un mensurando por dos vías complementarias: la ley de propagación de incertidumbres de la GUM (JCGM 100) y la propagación de distribuciones por Monte Carlo (JCGM 101). El estudiante debe ser capaz de construir el modelo de medición, evaluar contribuciones Tipo A y Tipo B, asignar funciones de densidad por máxima entropía, generar números pseudoaleatorios de calidad y reportar un intervalo de cobertura defendible, validando la aproximación analítica contra la simulación.

Temario

- Mensurando, modelo de medición y fuentes de incertidumbre.
- Evaluación Tipo A (estadística) y Tipo B (información previa).
- Ley de propagación de incertidumbres (LPU): coeficientes de sensibilidad, correlación, incertidumbre expandida y Welch–Satterthwaite.
- Funciones de densidad de probabilidad y principio de máxima entropía.
- Generación de números pseudoaleatorios: LCG, Mersenne Twister, PCG64; transformada inversa, Box–Muller, aceptación–rechazo.
- Método Monte Carlo para la propagación de distribuciones (JCGM 101): intervalo de cobertura, procedimiento adaptativo y validación de la GUM.
- Aplicación a seis problemas de ingeniería con incertidumbre lineal, no lineal, multiplicativa, correlacionada y de soporte acotado.

Propósito del tema. En las semanas anteriores cada simulación devolvía un número: una temperatura, una deflexión, una eficiencia. Ese número, sin una afirmación cuantitativa sobre su dispersión, es científicamente incompleto. Esta semana convierte cada resultado determinista en una variable aleatoria con una distribución, y enseña a responder la pregunta que todo comité doctoral formula: *¿cuál es la incertidumbre de ese valor y cómo la calculó?* El marco es metrológico (GUM), pero la herramienta de cálculo — Monte Carlo — es la misma que articula todo el curso.

Resultado de aprendizaje esperado

Al término de la semana el estudiante podrá: (i) escribir el modelo de medición $Y = f(X_1, \dots, X_n)$ de su propio proyecto e identificar cada fuente de incertidumbre como Tipo A o Tipo B; (ii) calcular la incertidumbre combinada por LPU incluyendo

correlaciones y reportar la incertidumbre expandida con su factor de cobertura; (iii) implementar la propagación Monte Carlo en Google Colab con un generador reproducible y construir el intervalo de cobertura por percentiles; y (iv) decidir, con un criterio numérico explícito, si la aproximación GUM es válida para su problema o si la no linealidad obliga a reportar el resultado Monte Carlo.

6.1. Del campo determinista al mensurando con incertidumbre

Un *mensurando* es la magnitud que se desea determinar. Rara vez se mide de forma directa: lo habitual es medir magnitudes de entrada $X_1, \dots, X_n$ y combinarlas mediante un *modelo de medición*

$$Y = f(X_1, X_2, \dots, X_n), \quad (174)$$

donde f puede ser una fórmula cerrada, el resultado de un ajuste o incluso la salida de una simulación numérica como las de las semanas previas. Cada X_i se conoce con una incertidumbre, descrita por una función de densidad de probabilidad (PDF) $g_{X_i}(\xi_i)$. La tarea metrológica es propagar esas PDF a través de f para obtener la PDF del mensurando $g_Y(\eta)$, de la cual se extraen una estimación (su esperanza) y una incertidumbre estándar (su desviación típica).

La GUM (*Guide to the Expression of Uncertainty in Measurement*, JCGM 100:2008) formaliza este programa. Su vocabulario es preciso y conviene fijarlo:

- **Estimación** x_i de la entrada X_i : el valor esperado $\mathbb{E}[X_i]$.
- **Incertidumbre estándar** $u(x_i)$: la desviación típica $\sqrt{\text{Var}[X_i]}$.
- **Incertidumbre estándar combinada** $u_c(y)$: la desviación típica de Y .
- **Incertidumbre expandida** $U = k u_c(y)$: define un intervalo $[y - U, y + U]$ con una probabilidad de cobertura declarada (típicamente 95 %).

La distinción esencial no es entre “errores aleatorios y sistemáticos” — terminología que la GUM abandona deliberadamente — sino entre *métodos de evaluación* de la incertidumbre: Tipo A y Tipo B.

6.2. Evaluación Tipo A y Tipo B de la incertidumbre estándar

6.2.1. Tipo A: evaluación estadística de una serie de observaciones

Cuando se dispone de m observaciones repetidas $q_1, \dots, q_m$ de una magnitud, la mejor estimación es la media y la incertidumbre estándar de esa media es

$$\bar{q} = \frac{1}{m} \sum_{j=1}^m q_j, \quad u_A(\bar{q}) = \frac{s(q)}{\sqrt{m}}, \quad s^2(q) = \frac{1}{m-1} \sum_{j=1}^m (q_j - \bar{q})^2, \quad (175)$$

con $\nu = m - 1$ grados de libertad. La evaluación Tipo A es, por tanto, la desviación estándar experimental de la media, y su fiabilidad crece con m . Un caso recurrente en el curso es la incertidumbre de los parámetros de un ajuste por mínimos cuadrados, que es Tipo A y hereda los grados de libertad $\nu = m - p$ (p parámetros).

6.2.2. Tipo B: evaluación a partir de información previa

Cuando no se dispone de repeticiones experimentales, la incertidumbre no puede estimarse mediante el análisis estadístico directo de una muestra. En este caso, la información proviene de fuentes previas, tales como certificados de calibración, especificaciones del fabricante, resolución instrumental, tolerancias geométricas, límites físicos, experiencia previa o juicio técnico fundamentado. A este procedimiento se le denomina evaluación de incertidumbre de Tipo B.

El criterio general consiste en asignar una función de densidad de probabilidad, denotada como PDF por sus siglas en inglés (*Probability Density Function*), que sea coherente con la información disponible. Una vez seleccionada la PDF, la incertidumbre estándar se toma como la desviación típica de dicha distribución:

$$u_B = \sigma_X,$$

donde X representa la magnitud de entrada asociada a la fuente de incertidumbre.

La elección de la PDF no debe ser arbitraria. Debe justificarse a partir de la información disponible. En ausencia de información adicional, se emplea el principio de máxima entropía, el cual selecciona la distribución menos comprometida, es decir, aquella que introduce la menor cantidad de supuestos adicionales compatibles con los datos conocidos. Por ejemplo, si sólo se conocen límites inferior y superior, la distribución rectangular es la opción menos informativa; si se conocen media y varianza sin límites físicos finitos, la distribución normal resulta apropiada.

A continuación se presentan distribuciones frecuentes en evaluaciones de incertidumbre de Tipo B y en simulaciones por Monte Carlo. En todos los casos, r , r_1 y r_2 representan números pseudoaleatorios independientes con distribución uniforme en el intervalo $(0, 1)$.

Distribuciones de probabilidad útiles para evaluación Tipo B y simulación Monte Carlo

En esta sección se describen distribuciones de probabilidad empleadas en la evaluación de incertidumbre Tipo B y en simulaciones por Monte Carlo. Para la generación pseudoaleatoria se considera que r , r_1 y r_2 son números pseudoaleatorios independientes con distribución uniforme en el intervalo $(0, 1)$. Asimismo, Z representa una variable

aleatoria normal estándar, es decir,

$$Z \sim \mathcal{N}(0, 1).$$

En la Figuras 39, 40, 41 y Figura 42 se presentan formas representativas de las PDF descritas en esta sección. Las curvas tienen propósito conceptual: no se busca comparar magnitudes verticales entre distribuciones, sino visualizar la forma cualitativa de cada modelo probabilístico y su relación con el tipo de información disponible.

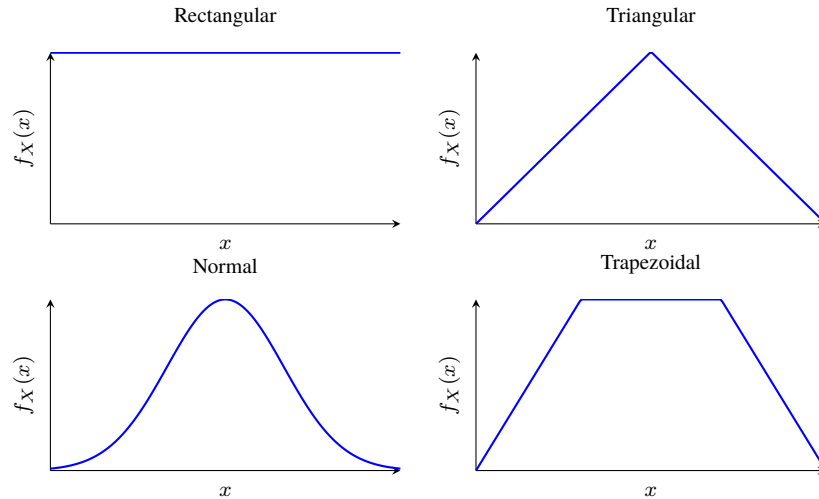


Figura 39: Formas representativas de distribuciones rectangular, triangular, normal y trapezoidal empleadas en evaluación de incertidumbre Tipo B.

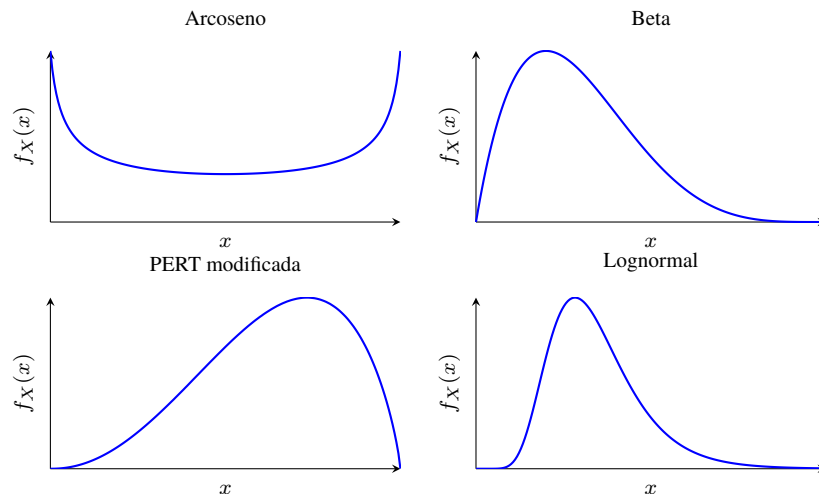


Figura 40: Formas representativas de distribuciones arcoseno, beta, PERT modificada y lognormal.

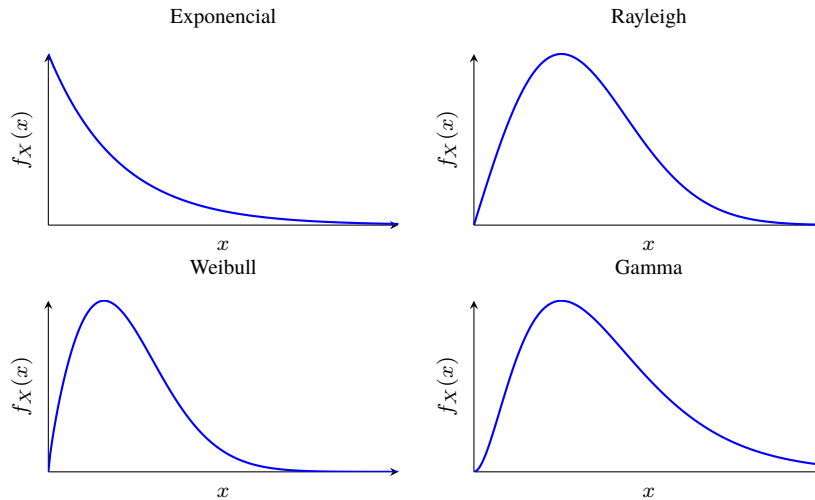


Figura 41: Formas representativas de distribuciones positivas empleadas en simulación Monte Carlo.

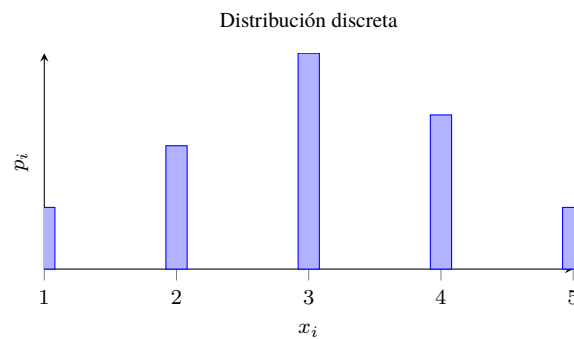


Figura 42: Forma representativa de una distribución discreta, donde la probabilidad se asigna a valores específicos x_i .

Distribución rectangular o uniforme

La distribución rectangular se emplea cuando sólo se conocen los límites simétricos $\pm a$ de una magnitud y no existe información que permita asignar mayor probabilidad a alguna región interna del intervalo. Es común en la evaluación de incertidumbre asociada a la resolución instrumental, redondeo numérico, tolerancias simples o especificaciones de fabricante expresadas únicamente mediante límites.

Los parámetros principales son el valor central x_0 y el semiancho del intervalo a . La incertidumbre estándar asociada se calcula como

$$u_B = \frac{a}{\sqrt{3}}.$$

Para simulación Monte Carlo, una realización pseudoaleatoria de la variable X puede

generarse mediante

$$X = x_0 + a(2r - 1).$$

Esta distribución representa el caso de máxima entropía cuando sólo se conocen los límites inferior y superior de la magnitud.

Distribución triangular simétrica

La distribución triangular simétrica se utiliza cuando se conocen los límites $\pm a$, pero se considera que los valores cercanos al centro son más probables que los valores extremos. Esta situación aparece cuando existe una estimación central razonable, aunque los límites extremos siguen siendo físicamente posibles.

Los parámetros son el valor central x_0 y el semiancho a . La incertidumbre estándar se obtiene como

$$u_B = \frac{a}{\sqrt{6}}.$$

La generación pseudoaleatoria puede realizarse mediante la transformación inversa de la función de distribución acumulada:

$$X = \begin{cases} x_0 + a(\sqrt{2r} - 1), & 0 < r < 1/2, \\ x_0 + a(1 - \sqrt{2(1-r)}), & 1/2 \leq r < 1. \end{cases}$$

Esta distribución es más restrictiva que la rectangular, dado que asigna mayor peso probabilístico a los valores cercanos al centro.

Distribución normal

La distribución normal se emplea cuando la información disponible proviene de un certificado de calibración, de un resultado experimental previo o de una fuente que reporta una incertidumbre expandida U asociada a un factor de cobertura k .

Los parámetros relevantes son el valor central x_0 , la incertidumbre expandida U y el factor de cobertura k . La incertidumbre estándar se calcula mediante

$$u_B = \frac{U}{k}.$$

Para la generación pseudoaleatoria se emplea

$$X = x_0 + u_B Z, \quad Z \sim \mathcal{N}(0, 1).$$

La distribución normal es apropiada cuando la magnitud resulta de la combinación de múltiples efectos pequeños e independientes, o cuando la fuente de información ya

proporciona una incertidumbre compatible con un modelo gaussiano.

Distribución trapezoidal simétrica

La distribución trapezoidal simétrica se emplea cuando se conocen límites extremos $\pm a$, pero existe una región central $\pm b$ donde la probabilidad puede considerarse aproximadamente constante. Esta distribución es intermedia entre la rectangular y la triangular.

Los parámetros son el valor central x_0 , el semiancho total a y el semiancho de la meseta central b , con

$$0 \leq b \leq a.$$

La incertidumbre estándar se calcula como

$$u_B = \sqrt{\frac{a^2 + b^2}{6}}.$$

Una forma práctica de generar valores pseudoaleatorios consiste en expresar la variable como la suma de dos variables uniformes independientes:

$$X = x_0 + Y_1 + Y_2,$$

donde

$$Y_1 \sim \mathcal{U}\left[-\frac{a-b}{2}, \frac{a-b}{2}\right], \quad Y_2 \sim \mathcal{U}\left[-\frac{a+b}{2}, \frac{a+b}{2}\right].$$

Esta distribución resulta útil cuando la información disponible indica que los valores centrales son aproximadamente equiprobables, pero los extremos no deben descartarse completamente.

Distribución arcoseno o forma U

La distribución arcoseno, también llamada distribución en forma U, se emplea cuando los valores cercanos a los extremos son más probables que los valores centrales. Aparece en errores periódicos, excentricidad, fase, vibración sinusoidal, errores angulares o desalineaciones con comportamiento armónico.

Los parámetros son el valor central x_0 y la amplitud a . La incertidumbre estándar se obtiene como

$$u_B = \frac{a}{\sqrt{2}}.$$

La generación pseudoaleatoria puede realizarse mediante

$$X = x_0 + a \sin\left[\pi \left(r - \frac{1}{2}\right)\right].$$

Esta distribución es relevante cuando el error puede modelarse como la proyección de una variable angular uniformemente distribuida.

Distribución beta

La distribución beta se emplea cuando la magnitud está restringida a un intervalo finito $[L, H]$, pero se desea controlar la forma de la distribución, su asimetría o su concentración dentro del intervalo. Es más flexible que las distribuciones rectangular, triangular o trapezoidal.

Los parámetros son el límite inferior L , el límite superior H y los parámetros de forma α y β . Si

$$B \sim \text{Beta}(\alpha, \beta),$$

entonces la variable escalada se define como

$$X = L + (H - L)B.$$

La incertidumbre estándar se calcula mediante

$$u_B = (H - L) \sqrt{\frac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)}}.$$

Esta distribución es útil cuando existe información previa suficiente para representar sesgo, concentración o asimetría dentro de límites físicos conocidos.

Distribución PERT modificada

La distribución PERT modificada se emplea cuando se conocen tres valores: un mínimo L , un máximo H y un valor más probable m . Es útil en estimaciones expertas, análisis de riesgo y evaluación de incertidumbre cuando existen límites físicos y una estimación preferente.

La distribución PERT puede formularse mediante una distribución beta escalada. Los parámetros de forma se definen como

$$\alpha = 1 + \lambda \frac{m - L}{H - L},$$

$$\beta = 1 + \lambda \frac{H - m}{H - L},$$

donde usualmente se toma

$$\lambda = 4.$$

La incertidumbre estándar se calcula como

$$u_B = (H - L) \sqrt{\frac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)}}.$$

La generación pseudoaleatoria se realiza mediante

$$X = L + (H - L)B, \quad B \sim \text{Beta}(\alpha, \beta).$$

Esta distribución permite incorporar el juicio experto de manera más estructurada que una distribución triangular simple.

Distribución lognormal

La distribución lognormal se emplea cuando la magnitud es estrictamente positiva y presenta dispersión multiplicativa. Es común en variables como rugosidad, tamaño de partícula, coeficientes positivos, factores de escala, tiempos positivos o parámetros físicos con variabilidad relativa.

Si

$$\ln X \sim \mathcal{N}(\mu, \sigma^2),$$

entonces la incertidumbre estándar de X está dada por

$$u_B = \sqrt{(e^{\sigma^2} - 1) e^{2\mu + \sigma^2}}.$$

La generación pseudoaleatoria se realiza mediante

$$X = \exp(\mu + \sigma Z), \quad Z \sim \mathcal{N}(0, 1).$$

Esta distribución no debe emplearse para variables que puedan tomar valores negativos, dado que su soporte está restringido a $X > 0$.

Distribución exponencial

La distribución exponencial se emplea para modelar el tiempo hasta un evento, una vida característica o procesos de espera con tasa constante. Su uso es común en confiabilidad, procesos estocásticos simples y fenómenos donde la probabilidad de ocurrencia por unidad de tiempo se mantiene constante.

El parámetro principal es la tasa $\lambda > 0$. La incertidumbre estándar se calcula como

$$u_B = \frac{1}{\lambda}.$$

La generación pseudoaleatoria puede realizarse mediante

$$X = -\frac{1}{\lambda} \ln(1 - r).$$

Esta distribución sólo toma valores positivos y resulta adecuada cuando el fenómeno presenta comportamiento sin memoria.

Distribución Rayleigh

La distribución Rayleigh se emplea para representar la magnitud resultante de dos componentes normales independientes de igual varianza. Aparece en errores radiales, amplitudes, desplazamientos planos, vibraciones o magnitudes resultantes en dos dimensiones.

El parámetro principal es la escala $\sigma > 0$. La incertidumbre estándar se calcula como

$$u_B = \sigma \sqrt{\frac{4 - \pi}{2}}.$$

La generación pseudoaleatoria se realiza mediante

$$X = \sigma \sqrt{-2 \ln(1 - r)}.$$

Esta distribución es adecuada cuando la variable analizada corresponde a una magnitud radial no negativa.

Distribución Weibull

La distribución Weibull se emplea en confiabilidad, fatiga, vida útil, falla de materiales y procesos donde la tasa de falla no es constante. Su flexibilidad permite representar comportamientos con tasa de falla creciente, decreciente o aproximadamente constante.

Los parámetros son la escala $\lambda > 0$ y la forma $k > 0$. La incertidumbre estándar se calcula mediante

$$u_B = \lambda \sqrt{\Gamma\left(1 + \frac{2}{k}\right) - \Gamma^2\left(1 + \frac{1}{k}\right)}.$$

La generación pseudoaleatoria se realiza mediante

$$X = \lambda [-\ln(1 - r)]^{1/k}.$$

La función $\Gamma(\cdot)$ corresponde a la función gamma de Euler.

Distribución gamma

La distribución gamma se utiliza para magnitudes positivas acumulativas, tiempos de proceso, energía, conteos continuos, suma de variables exponenciales o fenómenos donde la variable representa acumulación positiva.

Los parámetros son la forma $\alpha > 0$ y la escala $\theta > 0$. La incertidumbre estándar se calcula como

$$u_B = \sqrt{\alpha} \theta.$$

La generación pseudoaleatoria se expresa como

$$X \sim \text{Gamma}(\alpha, \theta),$$

y normalmente se realiza mediante generadores estándar disponibles en lenguajes científicos como MATLAB, Python o R.

Esta distribución es más flexible que la exponencial y permite modelar procesos positivos con diferentes grados de dispersión.

Distribución discreta

La distribución discreta se emplea cuando la magnitud sólo puede tomar valores específicos x_i con probabilidades asociadas p_i . Es útil en selección de modos, estados de operación, errores discretizados, clases de comportamiento o decisiones categóricas dentro de una simulación.

Las probabilidades deben satisfacer

$$\sum_i p_i = 1.$$

El valor esperado se calcula como

$$\bar{x} = \sum_i p_i x_i,$$

mientras que la incertidumbre estándar se obtiene mediante

$$u_B = \sqrt{\sum_i p_i (x_i - \bar{x})^2}.$$

Para generar valores pseudoaleatorios se define la función acumulada

$$F_j = \sum_{i=1}^j p_i.$$

Después, se selecciona el valor x_j tal que

$$F_{j-1} < r \leq F_j.$$

Esta formulación permite incorporar directamente información probabilística discreta dentro de una simulación Monte Carlo.

6.2.3. Criterio práctico de selección de la PDF

La selección de la función de densidad de probabilidad debe justificarse con base en la información disponible. Si sólo se conocen límites, la distribución rectangular es la opción más conservadora. Si existe un valor central más probable, puede emplearse una distribución triangular o PERT. Si la fuente proporciona incertidumbre expandida y factor de cobertura, se emplea una distribución normal. Si la variable es estrictamente positiva y presenta dispersión relativa, puede utilizarse una distribución lognormal, gamma o Weibull, según el fenómeno físico.

En consecuencia, la evaluación Tipo B no consiste únicamente en asignar una desviación estándar. El procedimiento requiere identificar la información disponible, seleccionar una PDF físicamente coherente, estimar sus parámetros y propagar la incertidumbre mediante el modelo de medición correspondiente.

En aplicaciones metrológicas, las distribuciones rectangular, triangular, normal, trapezoidal, arcoseno y beta son particularmente útiles para fuentes de incertidumbre de Tipo B. Las distribuciones lognormal, exponencial, Rayleigh, Weibull y gamma son más específicas, pero resultan convenientes cuando la magnitud de entrada está restringida a valores positivos o cuando el fenómeno físico tiene una interpretación probabilística particular.

Para simulación Monte Carlo, el procedimiento general es el siguiente:

1. Identificar cada magnitud de entrada X_i del modelo de medición.
2. Asociar a cada X_i una PDF justificada por la información disponible.
3. Generar N valores pseudoaleatorios $x_i^{(j)}$ para cada magnitud de entrada.
4. Evaluar el modelo de medición:

$$y^{(j)} = f\left(x_1^{(j)}, x_2^{(j)}, \dots, x_n^{(j)}\right), \quad j = 1, 2, \dots, N.$$

5. Estimar el valor esperado, la incertidumbre estándar y los intervalos de cobertura a partir de la muestra simulada de Y .

La ventaja del enfoque Monte Carlo es que permite propagar incertidumbres sin recurrir necesariamente a linealización local del modelo. Esto es especialmente importante cuando $f(\cdot)$ es no lineal, cuando existen límites físicos, cuando las distribuciones no son normales o cuando la combinación de incertidumbres produce una distribución de salida

asimétrica.

En consecuencia, la evaluación Tipo B no debe entenderse como una estimación débil o meramente subjetiva. Su validez depende de la correcta justificación de la PDF, de la trazabilidad de la información usada y de la consistencia física entre la fuente de incertidumbre y la distribución seleccionada.

6.3. Ley de propagación de incertidumbres

La GUM propaga la incertidumbre linealizando el modelo (174) en torno a las estimaciones $(x_1, \dots, x_n)$ mediante un desarrollo de Taylor de primer orden. Definiendo los *coeficientes de sensibilidad*

$$c_i = \left. \frac{\partial f}{\partial X_i} \right|_{x_1, \dots, x_n}, \quad (176)$$

la incertidumbre estándar combinada es, en ausencia de correlación entre las entradas,

$$u_c^2(y) = \sum_{i=1}^n c_i^2 u^2(x_i). \quad (177)$$

Cada término $c_i^2 u^2(x_i)$ es la contribución de la entrada i a la varianza del mensurando; su reparto porcentual es el *presupuesto de incertidumbre* (*uncertainty budget*), la herramienta de diagnóstico que indica dónde invertir para reducir la incertidumbre. Cuando las entradas están correlacionadas, la expresión completa incluye las covarianzas:

$$u_c^2(y) = \sum_{i=1}^n c_i^2 u^2(x_i) + 2 \sum_{i=1}^{n-1} \sum_{j=i+1}^n c_i c_j u(x_i, x_j), \quad u(x_i, x_j) = r_{ij} u(x_i) u(x_j), \quad (178)$$

donde $r_{ij} \in [-1, 1]$ es el coeficiente de correlación. Ignorar covarianzas no es conservador: una correlación positiva entre términos del mismo signo *aumenta* u_c , y una negativa la reduce. El Ejemplo 5 de esta semana muestra un caso donde despreciar la correlación subestima la incertidumbre por un factor mayor que dos.

6.3.1. Incertidumbre expandida y factor de cobertura

Para declarar un intervalo con probabilidad de cobertura dada, se multiplica u_c por un factor de cobertura k :

$$U = k u_c(y), \quad Y \in [y - U, y + U] \text{ con probabilidad } \approx p. \quad (179)$$

Bajo normalidad asintótica $k = 2$ corresponde a $p \approx 95.45\%$ y $k = 1.96$ a $p = 95\%$ exacto. Cuando los grados de libertad son finitos, el factor se toma del cuantil de la distribución t de Student con los grados de libertad *efectivos* dados por la fórmula de Welch–

Satterthwaite:

$$\nu_{\text{eff}} = \frac{u_c^4(y)}{\sum_{i=1}^n \frac{c_i^4 u^4(x_i)}{\nu_i}}, \quad k = t_{0.975}(\nu_{\text{eff}}). \quad (180)$$

Las contribuciones Tipo B basadas en límites se toman con $\nu_i \rightarrow \infty$ (información “perfectamente” conocida en su soporte), de modo que sólo las Tipo A con pocas repeticiones inflan k por encima de 2. El Ejemplo 1 ilustra el cálculo completo de ν_{eff} y su efecto sobre el intervalo.

6.4. Asignación de funciones de densidad de probabilidad

La propagación Monte Carlo exige algo que la LPU no necesita: la PDF completa de cada entrada, no sólo su media y varianza. La asignación se rige por el principio de máxima entropía y por el conocimiento físico del proceso. El catálogo operativo del curso aparece en la Tabla 27.

Tabla 27: Asignación de PDF a las magnitudes de entrada según la información disponible. La columna u da la incertidumbre estándar en función de los parámetros.

PDF	Información que codifica	u	Uso típico
Normal $\mathcal{N}(\mu, \sigma^2)$	media y varianza, soporte infinito	σ	Tipo A, certificados
Rectangular $\mathcal{U}(\mu \pm a)$	sólo límites, sin preferencia	$a/\sqrt{3}$	resolución, tolerancias
Triangular	límites con moda central	$a/\sqrt{6}$	suma de dos rectangulares
t de Student t_ν	media con pocos datos	$\sigma \sqrt{\frac{\nu}{\nu-2}}$	Tipo A con ν pequeño
Lognormal	magnitud positiva, asimétrica	$\mu \sqrt{e^{\sigma^2} - 1}$	resistencias de materiales
Beta sobre $[0, 1]$	proporción, soporte acotado	$\sqrt{\frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}}$	tasas de acierto

La asignación correcta es decisiva cuando el modelo es no lineal o el soporte es acotado. El Ejemplo 2 (modelo exponencial) muestra cómo PDF simétricas de entrada producen una PDF de salida sesgada; el Ejemplo 6 (tasa de acierto en $[0, 1]$) muestra cómo la normal de la GUM produce intervalos que invaden la región físicamente imposible > 1 , mientras la Beta respeta el soporte.

6.5. Generación de números pseudoaleatorios

Monte Carlo reduce la propagación de PDF a la generación de muestras. Un *generador de números pseudoaleatorios* (PRNG) produce una sucesión determinista que supera las pruebas estadísticas de aleatoriedad; “pseudo” significa que, fijada la semilla, la suce-

sión es reproducible — propiedad imprescindible para que una simulación doctoral sea verificable.

6.5.1. Generadores de base

El *generador lineal congruencial* (LCG), $x_{n+1} = (ax_n + c) \bmod M$, es el arquetipo histórico: rápido pero de período corto y con correlaciones seriales que lo descalifican para trabajo serio. El *Mersenne Twister* (MT19937) lo reemplazó por su período colosal $2^{19937} - 1$ y su equidistribución en 623 dimensiones, y fue durante dos décadas el estándar de facto. NumPy adopta hoy por defecto el *PCG64* (*permuted congruential generator*), que combina un LCG con una permutación de salida para lograr mejor calidad estadística, menor estado y flujos múltiples independientes. La práctica recomendada en Colab es instanciar un generador explícito con semilla:

```
rng = np.random.default_rng(semilla),
```

y *nunca* usar las funciones globales `np.random.*`, cuyo estado compartido rompe la reproducibilidad en cuanto el código crece.

6.5.2. De uniforme a cualquier distribución

Todo PRNG entrega uniformes $U \sim \mathcal{U}(0, 1)$. Para muestrear una PDF arbitraria se usan tres mecanismos:

- **Transformada inversa.** Si F es la función de distribución acumulada y $U \sim \mathcal{U}(0, 1)$, entonces $X = F^{-1}(U)$ tiene distribución F . Es exacta y eficiente cuando F^{-1} es conocida (exponencial, Weibull, triangular).
- **Box–Muller.** Para la normal, $Z_1 = \sqrt{-2 \ln U_1} \cos(2\pi U_2)$ y $Z_2 = \sqrt{-2 \ln U_1} \sin(2\pi U_2)$ generan dos normales estándar independientes a partir de dos uniformes.
- **Aceptación–rechazo.** Para PDF sin inversa cerrada: se muestrea de una propuesta $h(x)$ que acota $g(x) \leq c h(x)$ y se acepta con probabilidad $g(x)/(c h(x))$. Es general pero su eficiencia cae si la cota c es holgada.

En la práctica del curso estas primitivas están encapsuladas en `rng.normal`, `rng.uniform`, `rng.lognormal`, etc.; conocer su fundamento es lo que permite diagnosticar cuándo una muestra está mal generada.

6.6. Método Monte Carlo para la propagación de distribuciones

El Suplemento 1 de la GUM (JCGM 101:2008) define la propagación de distribuciones por Monte Carlo (MCM) como alternativa — y verificación — de la LPU. El procedimiento es directo:

1. Asignar a cada entrada X_i su PDF g_{X_i} .
2. Para cada una de M pruebas, muestrear un vector $(\xi_1^{(r)}, \dots, \xi_n^{(r)})$ y evaluar $\eta^{(r)} = f(\xi_1^{(r)}, \dots, \xi_n^{(r)})$.
3. La muestra $\{\eta^{(r)}\}_{r=1}^M$ es una representación discreta de g_Y . La estimación es su media, la incertidumbre estándar es su desviación típica, y el intervalo de cobertura del $p\%$ son los percentiles $[(1-p)/2, (1+p)/2]$.

La fortaleza del MCM es que *no linealiza*: propaga la no linealidad y la asimetría exactas de f , captura soportes acotados y no presupone normalidad de la salida. Su precisión está gobernada por el error de muestreo, que decae como

$$u(\hat{\theta}_{MC}) \propto \frac{\sigma_Y}{\sqrt{M}}, \quad (181)$$

es decir, $O(M^{-1/2})$: reducir el error a la mitad cuesta cuadruplicar M . El procedimiento adaptativo de la JCGM 101 fija M controlando que la última cifra significativa del resultado sea numéricamente estable.

6.6.1. Validación de la GUM mediante Monte Carlo

La JCGM 101 propone usar el MCM como juez de la LPU. Sea $[y_{\text{low}}, y_{\text{high}}]$ el intervalo GUM y $[y_{\text{low}}^{\text{MC}}, y_{\text{high}}^{\text{MC}}]$ el intervalo Monte Carlo, ambos al mismo nivel de cobertura. Se define una tolerancia numérica δ a partir del número de cifras significativas con que se desea reportar el resultado, y la GUM se considera *validada* si

$$|y_{\text{low}} - y_{\text{low}}^{\text{MC}}| \leq \delta \quad \text{y} \quad |y_{\text{high}} - y_{\text{high}}^{\text{MC}}| \leq \delta. \quad (182)$$

Cuando la validación falla, el modelo es suficientemente no lineal o asimétrico para que la linealización de la LPU sea inadecuada, y *debe* reportarse el resultado Monte Carlo. Los seis ejemplos siguientes recorren deliberadamente el espectro: desde un caso casi lineal donde GUM y MC coinciden (Ejemplo 1) hasta casos donde la GUM falla la validación por no linealidad (Ejemplo 2), por no normalidad en la cola (Ejemplo 3) o por soporte acotado (Ejemplo 6).

6.7. Ejemplo 1 — Calibración de un sensor: el caso canónico GUM con validación Monte Carlo

6.7.1. Planteamiento

Un sistema de calibración de sensores requiere caracterizar un transductor de presión cuya respuesta nominal es lineal. Se aplican $m = 10$ presiones de referencia conocidas (patrón con incertidumbre Tipo B $u_{\text{ref}} = 0.1155$ kPa, distribución rectangular de

semiancho 0.2 kPa) y se registra la salida en mV. El modelo de medición es la recta de calibración inversa: dada una lectura y_0 del sensor, estimar la presión

$$x_0 = \frac{y_0 - a}{b}, \quad (183)$$

donde a (intersección) y b (pendiente) provienen de un ajuste por mínimos cuadrados ordinarios. Este es el problema metrológico arquetípico: combina una incertidumbre Tipo A (los parámetros del ajuste, *correlacionados* entre sí), una Tipo A de repetibilidad de la lectura y una Tipo B del patrón. El mensurando es x_0 y su modelo (183) es no lineal en los parámetros, lo que lo hace ideal para confrontar LPU y Monte Carlo.

Tabla 28: Parámetros del ajuste de calibración y sus incertidumbres. La correlación pendiente–intersección es intrínseca al ajuste y no puede despreciarse.

Magnitud	Estimación	Incert. estándar	Tipo
Intersección a	1.9185 mV	0.1051 mV	A
Pendiente b	0.501 84 mV/kPa	0.001 78 mV/kPa	A
Correlación r_{ab}	−0.845	—	A
Desv. residual s	0.1863 mV	($\nu = 9$)	A
Repetibilidad lectura $u(y_0)$	—	0.0501 mV	A
Patrón de referencia u_{ref}	—	0.1155 kPa	B (rect.)

6.7.2. Propagación analítica (LPU)

Los coeficientes de sensibilidad de (183) son $\frac{\partial x_0}{\partial a} = -1/b$, $\frac{\partial x_0}{\partial b} = -(y_0 - a)/b^2 = -x_0/b$ y $\frac{\partial x_0}{\partial y_0} = 1/b$. La incertidumbre combinada incorpora la covarianza $u(a, b) = r_{ab} u(a) u(b)$ según (178), y se suma en cuadratura con la contribución del patrón. El cálculo da $u_c(x_0) = 0.1893$ kPa. Con grados de libertad efectivos $\nu_{\text{eff}} = 45.2$ por Welch–Satterthwaite (180), el factor de cobertura es $k = t_{0.975}(45.2) = 2.014$ (ligeramente superior al valor asintótico 1.96), de donde $U = 0.3812$ kPa y el intervalo GUM al 95 % es $[49.597, 50.360]$ kPa para una lectura que estima $x_0 = 49.979$ kPa.

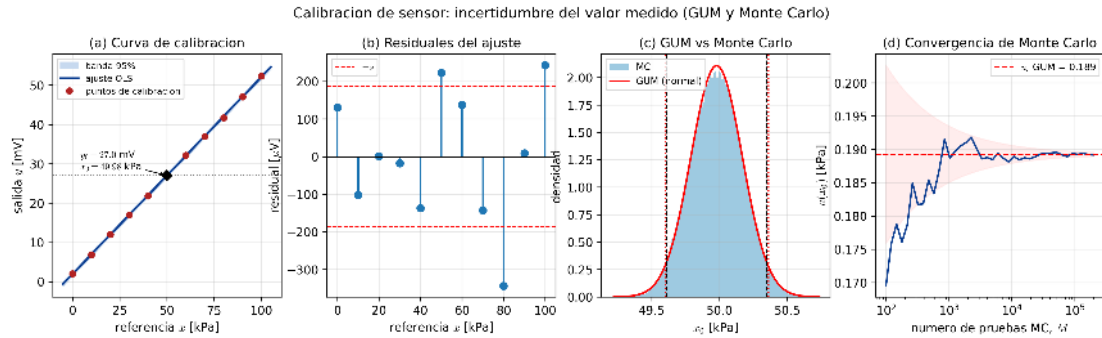


Figura 43: Calibración de sensor con análisis de incertidumbre. Panel 1: datos, recta ajustada y banda de predicción. Panel 2: histograma Monte Carlo del mensurando x_0 con el intervalo de cobertura del 95 % y la estimación GUM superpuesta. Panel 3: presupuesto de incertidumbre (reparto de la varianza por fuente). Panel 4: comparación de los intervalos GUM ($k u_c$, Ec. 179) y Monte Carlo (percentiles). Resultados del Código 1 del Anexo de esta semana.

6.7.3. Resultados y lectura metodológica

1. **GUM y Monte Carlo coinciden: el modelo es casi lineal.** La propagación Monte Carlo con $M = 2 \times 10^5$ pruebas da media 49.979 kPa, incertidumbre 0.1892 kPa e intervalo $[49.612, 50.346]$. La diferencia con el intervalo GUM es de apenas 0.014 kPa en cada extremo, atribuible al factor t de Welch–Satterthwaite que la GUM aplica y el MC no necesita. Para fines de ingeniería ambos métodos concuerdan: este es el régimen donde la LPU es plenamente válida y el reporte analítico es suficiente.
2. **El presupuesto de incertidumbre ordena las prioridades.** El reparto de la varianza es: patrón de referencia 37.2 %, regresión (parámetros a, b correlacionados) 35.0 %, repetibilidad de la lectura 27.7 % y resolución 0.1 %. La lección de gestión es inmediata: para reducir la incertidumbre de esta calibración no sirve promediar más lecturas (la repetibilidad ya es secundaria); hay que conseguir un *patrón mejor y más puntos de calibración* bien distribuidos.
3. **La correlación de los parámetros es ineludible.** En todo ajuste lineal a y b están correlacionados ($r_{ab} = -0.845$ aquí). Tratar $u(a)$ y $u(b)$ como independientes habría dado una incertidumbre incorrecta; el término de covarianza de (178) es parte estructural del problema, no un refinamiento opcional.

6.8. Ejemplo 2 — Elevación de temperatura por cortocircuito en un cable OPGW: no linealidad y probabilidad de falla

6.8.1. Planteamiento

Un cable OPGW (*optical ground wire*) debe soportar la corriente de cortocircuito de la línea sin que su temperatura exceda el límite del aluminio (200 °C). El modelo adiabático de la norma IEC 60949 relaciona la temperatura final θ_f con la corriente I , la duración t , la sección S y propiedades del material:

$$\theta_f = (\theta_i + \beta) \exp\left(\frac{I^2 t}{K^2 S^2}\right) - \beta, \quad (184)$$

donde θ_i es la temperatura inicial, K una constante del material y β el recíproco del coeficiente de temperatura de la resistividad. El modelo es *fuertemente no lineal* (exponencial en $I^2 t$), lo que anticipa que la LPU — que linealiza — será inadecuada. Las entradas y sus PDF aparecen en la Tabla 29; el mensurando es θ_f y la magnitud de decisión es la probabilidad de exceder 200 °C.

Tabla 29: Magnitudes de entrada del modelo IEC 60949 (Ec. 184) y sus distribuciones asignadas por máxima entropía.

Magnitud	Estimación	Incert. estándar	PDF
Corriente I	12.5 kA	4 %	Normal
Duración t	0.5 s	0.0346 s	Rectangular (± 0.06)
Sección S	90 mm ²	2 %	Rectangular
Temp. inicial θ_i	40 °C	8.66 °C	Rectangular (± 15)
Constante K	148	1.5	Normal
Parámetro β	228	3	Normal

6.8.2. Resultados y lectura metodológica

El valor nominal es $\theta_f = 188.3$ °C, cómodamente por debajo del límite. Pero la pregunta de ingeniería no es el valor nominal sino el riesgo de excederlo.

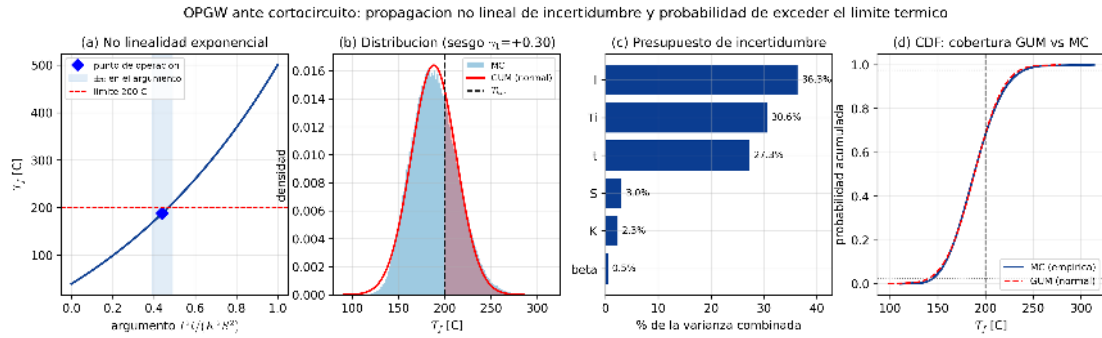


Figura 44: Cortocircuito en OPGW. Panel 1: PDF Monte Carlo de θ_f con el límite de 200 °C y la masa de probabilidad de falla sombreada. Panel 2: comparación de la PDF normal supuesta por la GUM contra el histograma MC, evidenciando la asimetría positiva. Panel 3: presupuesto de incertidumbre. Panel 4: convergencia de la probabilidad de falla con el número de pruebas M . Resultados del Código 2 del Anexo de esta semana.

1. **La GUM no se valida: la no linealidad genera asimetría.** La LPU da $u_c = 24.32$ °C y un intervalo simétrico $[139.6, 236.9]$. El Monte Carlo ($M = 5 \times 10^5$) da media 189.2 °C, incertidumbre 24.58 °C e intervalo $[145.0, 240.5]$, con *asimetría positiva* (*skewness* +0.298). El extremo inferior difiere en 5.4 °C y el superior en 3.6 °C, muy por encima de la tolerancia $\delta = 0.5$ °C: la GUM **falla la validación**. La exponencial estira la cola superior, y la media MC supera al valor nominal en +0.90 °C — un sesgo que la LPU, centrada por construcción, no puede captar.
2. **La probabilidad de falla es la cifra de diseño.** El MC estima $P(\theta_f > 200$ °C) = 31.80 % $\pm$ 0.09 %. Un valor nominal 11.7 °C por debajo del límite oculta que casi un tercio de las realizaciones lo exceden: el margen determinista es engañoso cuando la incertidumbre es grande y el modelo no lineal. Reportar sólo $\theta_f = 188$ °C sería un error de criterio profesional.
3. **El presupuesto señala la corriente.** El reparto de la varianza es I 36.3 %, θ_i 30.6 %, t 27.3 %, S 3.0 %, K 2.3 % y β 0.5 %. La corriente domina porque entra al cuadrado y dentro de la exponencial; su coeficiente de sensibilidad amplifica una incertidumbre relativa modesta (4 %). Reducir la incertidumbre de la estimación de la corriente de cortocircuito de la línea es la acción de mayor retorno.

6.9. Ejemplo 3 — Confiabilidad estructural de losas: índice de confiabilidad y no normalidad en la cola

6.9.1. Planteamiento

Se comparan dos sistemas de entrepiso — losa maciza y losa reticular (aligerada con casetones) — bajo el estado límite de servicio por deflexión. La deflexión máxima de una

losa apoyada se modela como

$$\delta = \frac{\alpha w L^4}{EI}, \quad \delta_{\text{adm}} = \frac{L}{360}, \quad (185)$$

con $\alpha = 5/384$, claro L , carga distribuida $w = w_d + w_l$ (muerta más viva), módulo $E = 4700\sqrt{f'_c}$ y momento de inercia I . Las fuentes de incertidumbre son la resistencia del concreto f'_c (lognormal, COV 12 %, que propaga a E), la carga viva w_l (COV 25 %), el claro L (± 0.02 m) y la inercia I (COV 12 % en la maciza, 15 % en la reticular por la variabilidad geométrica del casetón). El mensurando es δ y la magnitud de decisión es la probabilidad de falla $P(\delta > \delta_{\text{adm}})$ y el índice de confiabilidad $\beta = -\Phi^{-1}(P_f)$.

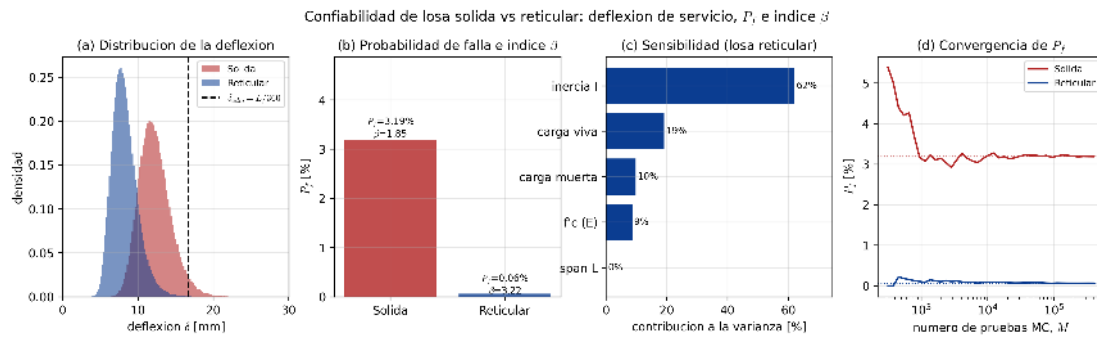


Figura 45: Confiabilidad de losas maciza vs reticular. Panel 1: PDF Monte Carlo de la deflexión de ambos sistemas con el límite admisible δ_{adm} . Panel 2: probabilidad de falla e índice de confiabilidad β . Panel 3: sensibilidad (reparto de varianza) del sistema reticular. Panel 4: comparación del β por Monte Carlo y por FORM (aproximación normal de primer orden). Resultados del Código 3 del Anexo de esta semana.

6.9.2. Resultados y lectura metodológica

- 1. La losa aligerada es más ligera y más confiable.** La maciza deflcta $\delta_0 = 12.02$ mm (72.1 % del admisible 16.67 mm) con $P_f = 3.186\% \pm 0.028\%$ y $\beta_{\text{MC}} = 1.854$. La reticular deflcta $\delta_0 = 8.04$ mm (48.3 % del admisible) con $P_f = 0.064\% \pm 0.004\%$ y $\beta_{\text{MC}} = 3.223$. El resultado es contraintuitivo sólo en apariencia: al retirar peso propio en la zona de casetones, la carga muerta baja más de lo que baja la inercia, y el sistema gana margen. La confiabilidad no se lee del valor nominal sino de la distancia de la cola a la frontera.
- 2. FORM y Monte Carlo divergen en la cola.** El índice por FORM (que asume normalidad y linealiza en el punto de diseño) da $\beta_{\text{FORM}} = 2.076$ (maciza) y 4.927 (reticular), frente a los valores MC 1.854 y 3.223. La discrepancia crece con β porque la PDF de δ es no normal — producto y cociente de variables lognormales y normales —, y FORM sobreestima la confiabilidad justo donde más importa: en eventos raros. Para P_f pequeñas, el Monte Carlo (o un FORM/SORM cuidadoso)

es obligatorio; la aproximación gaussiana es optimista.

3. **El presupuesto identifica la inercia como palanca.** En la reticular, la varianza se reparte en inercia I 62 %, carga viva 19 %, carga muerta 10 %, f'_c (vía E) 9 % y claro 0 %. La incertidumbre geométrica del casetón domina: controlar la tolerancia de fabricación de los aligeramientos reduce la incertidumbre estructural más que mejorar el concreto. El claro, pese a entrar a la cuarta potencia, contribuye de forma despreciable porque su incertidumbre relativa es mínima.

6.10. Ejemplo 4 — Optimización de una bomba centrífuga bajo incertidumbre: modelo multiplicativo y BEP

6.10.1. Planteamiento

La eficiencia hidráulica de una bomba centrífuga se obtiene del balance entre la potencia hidráulica entregada y la potencia al eje:

$$\eta = \frac{\rho g Q H}{P}, \quad (186)$$

con ρ la densidad del fluido, g la gravedad, Q el caudal, H la altura manométrica y P la potencia al eje. Este es el arquetipo de *modelo multiplicativo*: todas las variables entran como producto o cociente. La consecuencia metodológica es directa — al tomar logaritmo y derivar, la LPU (177) se reduce a una suma en cuadratura de incertidumbres relativas:

$$\left(\frac{u_c(\eta)}{\eta} \right)^2 = \left(\frac{u(Q)}{Q} \right)^2 + \left(\frac{u(H)}{H} \right)^2 + \left(\frac{u(P)}{P} \right)^2 + \left(\frac{u(\rho)}{\rho} \right)^2. \quad (187)$$

El objetivo de ingeniería no es sólo cuantificar $u(\eta)$ en un punto, sino propagar la incertidumbre a una *optimización*: localizar el punto de mejor eficiencia (BEP, *best efficiency point*) sobre la curva $\eta(Q)$ y reportar la incertidumbre tanto del caudal óptimo Q_{BEP} como de la eficiencia máxima η_{max} .

Tabla 30: Incertidumbres relativas de instrumentación de la bomba (Tipo B salvo indicación). El modelo multiplicativo (187) sólo requiere las incertidumbres relativas.

Magnitud	Incert. relativa	Distribución
Caudal Q (caudalímetro)	2.0 %	Normal
Altura H (transductores de presión)	1.5 %	Normal
Potencia al eje P (par-velocidad)	1.5 %	Normal
Densidad ρ	0.03 %	Rectangular

6.10.2. Propagación analítica y optimización

En un punto de operación representativo, (187) da una incertidumbre relativa combinada $u_c(\eta)/\eta = 2.916\%$, dominada por el caudal. Para localizar el BEP se ajusta una parábola a la curva $\eta(Q)$ medida y se propaga la incertidumbre de los puntos a los coeficientes del ajuste, y de éstos al vértice $Q_{\text{BEP}} = -c_1/(2c_2)$ y a η_{max} . La propagación se realiza por Monte Carlo porque el vértice es una función no lineal (cociente) de los coeficientes ajustados.

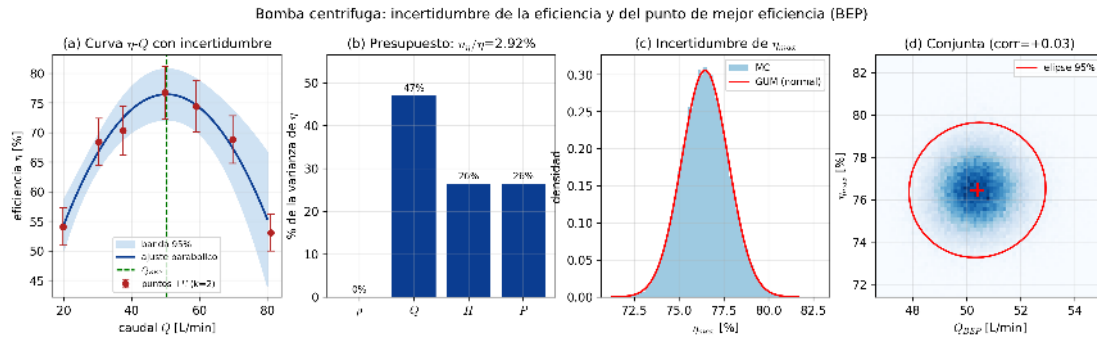


Figura 46: Optimización de bomba centrífuga bajo incertidumbre. Panel 1: curvas $\eta(Q)$ Monte Carlo con la banda de incertidumbre y el BEP. Panel 2: presupuesto de incertidumbre relativa de η (Ec. 187). Panel 3: distribución conjunta de Q_{BEP} y η_{max} propagada por Monte Carlo. Panel 4: intervalos de cobertura del caudal óptimo y de la eficiencia máxima. Resultados del Código 4 del Anexo de esta semana.

6.10.3. Resultados y lectura metodológica

1. **En un modelo multiplicativo manda la peor incertidumbre relativa.** El reparto de varianza es caudal 47.1 %, altura 26.5 %, potencia 26.5 % y densidad 0.0 %. El caudal domina no porque su incertidumbre sea grande en absoluto, sino porque entra al cuadrado en (187) con el mayor valor relativo. Mejorar el caudalímetro del 2 % al 1 % reduciría la incertidumbre combinada más que cualquier otra acción individual.
2. **La incertidumbre se propaga al óptimo, no sólo al punto.** El BEP estimado es $Q_{\text{BEP}} = 50.33 \text{ L/min}$ con $\eta_{\text{max}} = 76.46\%$. La propagación Monte Carlo da $Q_{\text{BEP}} = 50.39 \pm 1.03 \text{ L/min}$ y $\eta_{\text{max}} = 76.45 \pm 1.30\%$ (incertidumbres estándar); con factor $k = 2$, el caudal óptimo se conoce a $\pm 2.06 \text{ L/min}$ y la eficiencia máxima a ± 2.61 puntos. Reportar el BEP sin su incertidumbre es afirmar una precisión que los instrumentos no sustentan.
3. **Caudal óptimo y eficiencia máxima son casi independientes.** La correlación propagada es $\text{corr}(Q_{\text{BEP}}, \eta_{\text{max}}) = +0.029$, prácticamente nula. Esto justifica reportar ambos intervalos por separado sin una elipse de confianza conjunta, una simplificación válida *sólo* porque la simulación confirmó la baja correlación; no es

un supuesto que pueda darse por sentado a priori.

6.11. Ejemplo 5 — Velocimetría vehicular por flujo óptico desde UAV: incertidumbre correlacionada

6.11.1. Planteamiento

Un sistema de gestión de tránsito estima la velocidad de vehículos a partir del flujo óptico medido en video aéreo capturado por un UAV. La cadena fotogramétrica convierte el desplazamiento en píxeles Δp entre cuadros a velocidad física:

$$v = \frac{\Delta p \cdot p_{\text{size}} \cdot A}{f \cdot \Delta t}, \quad (188)$$

donde p_{size} es el tamaño físico del píxel del sensor, A la altura de vuelo, f la distancia focal y Δt el periodo entre cuadros. El factor $\text{GSD} = p_{\text{size}} A / f$ es la resolución espacial en el suelo (*ground sampling distance*). El punto metodológico central es la *estructura de la incertidumbre*: la altura A , la focal f y el periodo Δt son los mismos para todos los vehículos medidos en una pasada — son fuentes *sistemáticas correlacionadas* — mientras que Δp es independiente vehículo a vehículo (*aleatoria*).

Tabla 31: Parámetros de la cadena fotogramétrica del UAV. La distinción sistemática/aleatoria gobierna cómo se combina la incertidumbre sobre una flota de vehículos.

Magnitud	Valor	Incert. estándar	Carácter
Altura de vuelo A	80 m	2.5 m	Sistemático
Distancia focal f	8.8 mm	0.05 mm	Sistemático
Periodo entre cuadros Δt	33.33 ms	0.5 ms	Sistemático
Tamaño de píxel p_{size}	3.45 μm	—	Fijo
Desplazamiento Δp	9.2–31.0 px	0.4 px	Aleatorio

6.11.2. Propagación analítica (LPU)

Con los valores de la Tabla 31 la resolución es $\text{GSD} = 3.136 \text{ cm/px}$ y la incertidumbre *sistemática* relativa, común a todos los vehículos, es

$$\frac{u_{\text{sys}}}{v} = \sqrt{\left(\frac{u(A)}{A}\right)^2 + \left(\frac{u(f)}{f}\right)^2 + \left(\frac{u(\Delta t)}{\Delta t}\right)^2} = 3.513 \%. \quad (189)$$

Al promediar la velocidad de $N = 8$ vehículos, la componente *aleatoria* de Δp se reduce como $1/\sqrt{N}$, pero la componente sistemática *no se promedia*: afecta por igual a todas las mediciones y permanece. Ignorar la correlación lleva al error clásico de creer que medir más vehículos reduce indefinidamente la incertidumbre de la velocidad media.

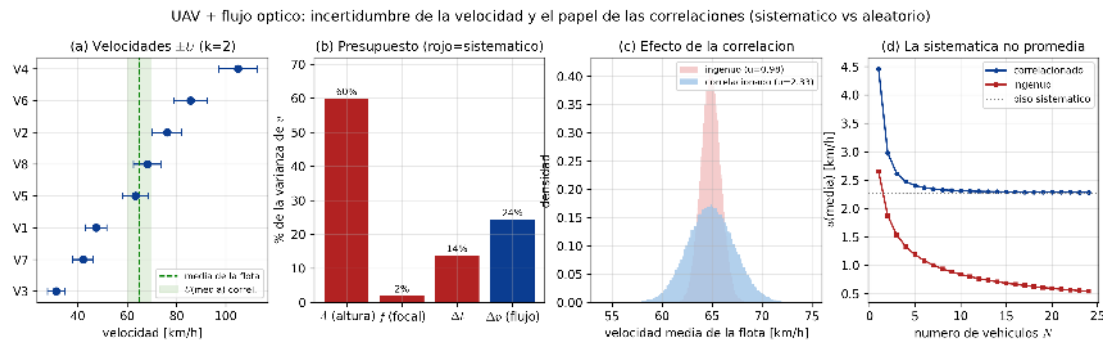


Figura 47: Velocimetría por flujo óptico desde UAV con incertidumbre correlacionada. Panel 1: velocidad estimada por vehículo con barras de incertidumbre. Panel 2: descomposición sistemática/aleatoria de la incertidumbre (Ec. 189). Panel 3: distribución Monte Carlo de la velocidad media de la flota bajo los modelos correlacionado e independiente. Panel 4: factor de subestimación al ignorar la correlación. Resultados del Código 5 del Anexo de esta semana.

6.11.3. Resultados y lectura metodológica

1. **La velocidad media de la flota es 64.87 km/h, pero su incertidumbre depende del modelo.** Tratando correctamente la correlación, $u(\bar{v}) = 2.328$ km/h. Bajo el modelo ingenuo que supone las ocho mediciones independientes, $u(\bar{v}) = 0.979$ km/h. El Monte Carlo reproduce ambos valores (2.327 y 0.979), confirmando que la diferencia es estructural y no un artefacto numérico.
2. **Ignorar la correlación subestima la incertidumbre por un factor 2.38.** El modelo ingenuo reporta menos de la mitad de la incertidumbre real. En un sistema de tránsito con consecuencias legales (infracciones por exceso de velocidad), esa subestimación es inadmisibles: el intervalo de cobertura sería demasiado estrecho y produciría falsos positivos. La componente sistemática del 3.513 % es un *piso* de incertidumbre que ningún promedio sobre vehículos puede atravesar.
3. **La calibración del UAV pesa más que el número de vehículos.** Como la incertidumbre sistemática no se promedia, la vía para reducir $u(\bar{v})$ no es medir más vehículos sino calibrar mejor la altura de vuelo, la focal y el sincronismo de cuadros. Medir Δp con sub-píxel (0.4 px) ya es secundario frente al 3.5 % sistemático.

6.12. Ejemplo 6 — Exactitud de un sistema RAG industrial: incertidumbre de soporte acotado

6.12.1. Planteamiento

Un sistema RAG (*retrieval-augmented generation*) para aplicaciones industriales se evalúa como una cascada de dos etapas: recuperación y generación. La exactitud extremo

a extremo es el producto de las exactitudes condicionales:

$$a_{e2e} = a_{\text{ret}} \cdot a_{\text{gen}}, \quad (190)$$

donde a_{ret} es la fracción de consultas con recuperación correcta y a_{gen} la fracción de respuestas correctas *dado* que la recuperación fue exitosa. Cada exactitud se estima de una muestra finita de evaluación (Tipo A binomial), y aquí aparece el reto metodológico distintivo: la magnitud está *acotada* en $[0, 1]$. Cuando la exactitud es alta y la muestra pequeña, el intervalo normal de la LPU puede sobrepasar el límite físico 1.0, produciendo un resultado sin sentido.

Tabla 32: Resultados de la evaluación del sistema RAG. La exactitud de generación se mide condicionada a recuperación exitosa.

Etapas	Aciertos/Total	Estimación	Incert. (u)
Recuperación a_{ret}	54/60	0.9000	0.0387
Generación a_{gen}	46/48	0.9583	0.0288
Extremo a extremo a_{e2e}	—	0.8625	0.0453

6.12.2. Propagación analítica y su falla en el borde

Por ser (190) multiplicativa, la LPU combina incertidumbres relativas y da $u(a_{e2e}) = 0.0453$, con intervalo GUM al 95 % $[0.7737, 0.9513]$. El síntoma de que la aproximación normal es inadecuada aparece al examinar la componente $a_{\text{gen}} = 0.9583$: su intervalo normal al 95 % tiene límite superior 1.0149, es decir, asigna probabilidad a una exactitud *mayor que el 100 %*, lo cual es físicamente imposible. La causa es la asimetría del binomial cerca del borde, que la gaussiana no captura.

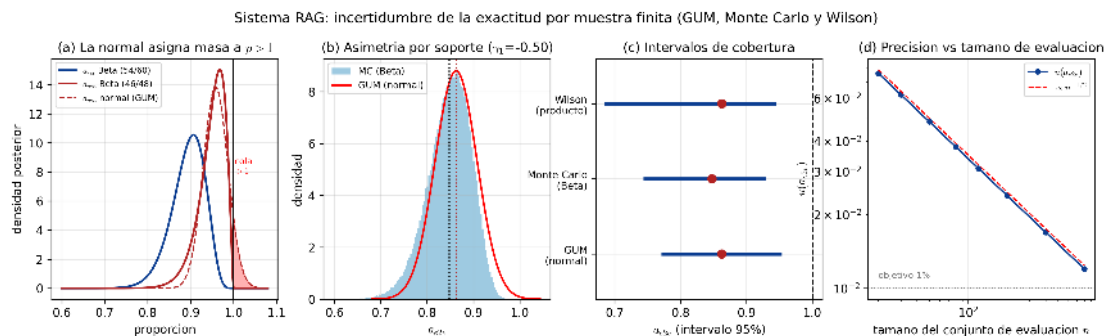


Figura 48: Incertidumbre de exactitud de un sistema RAG con soporte acotado. Panel 1: posteriores Beta de cada etapa y el producto Monte Carlo. Panel 2: comparación de intervalos GUM (normal), Monte Carlo (Beta) y Wilson. Panel 3: la cola normal de a_{gen} rebasa el límite físico 1.0. Panel 4: proyección a 1000 consultas con su intervalo de cobertura. Resultados del Código 6 del Anexo de esta semana.

6.12.3. Resultados y lectura metodológica

1. **El soporte acotado obliga a abandonar la normal.** El Monte Carlo correcto propaga distribuciones *Beta* (posterior de Jeffreys para cada binomial), garantizando que toda muestra cae en $[0, 1]$. El resultado es $a_{e2e} = 0.8478$ con $u = 0.0465$, intervalo $[0.7461, 0.9268]$ y asimetría -0.50 . La cola izquierda es más larga: la exactitud puede ser bastante peor de lo nominal con mayor probabilidad que mejor, información que el intervalo simétrico GUM oculta.
2. **El intervalo normal pierde validez física en el borde.** Que el límite superior de a_{gen} sea 1.0149 no es un detalle estético: invalida el reporte. El método Monte Carlo con *Beta*, o un intervalo de Wilson para el producto ($[0.6869, 0.9424]$), respetan la restricción $[0, 1]$ por construcción. Para exactitudes altas con muestras de evaluación pequeñas — la situación normal en validación de sistemas de IA — la GUM gaussiana *no* es defendible.
3. **La proyección operativa traduce la incertidumbre a unidades de negocio.** Sobre 1000 consultas en producción, el modelo predice 862 respuestas correctas con intervalo de cobertura Monte Carlo del 95 % $[746, 927]$. Ese rango de ± 90 consultas es el insumo cuantitativo para dimensionar revisión humana y acuerdos de nivel de servicio. La incertidumbre deja de ser un número abstracto y se vuelve una decisión de capacidad.

6.13. Síntesis de la semana

Los seis ejemplos recorren deliberadamente el espectro de comportamientos de la incertidumbre, de modo que cada estudiante reconozca a cuál pertenece su propio mensurando. La Tabla 33 resume el mapa.

Tabla 33: Mapa metodológico de la semana. Cada ejemplo ilustra un régimen distinto de propagación de incertidumbre y un criterio distinto sobre la validez de la GUM frente al Monte Carlo.

Ej.	Tipo de modelo	Reto dominante	Veredicto GUM vs MC
1	Lineal (regresión inversa)	Parámetros correlacionados	GUM válida; coincide con MC
2	Exponencial no lineal	No linealidad fuerte, asimetría	GUM <i>no</i> validada; usar MC
3	Producto/cociente	No normalidad en la cola	FORM optimista; MC obligatorio
4	Multiplicativo	Incertidumbre del óptimo (BEP)	MC para el vértice no lineal
5	Lineal con correlación	Sistemática no promediable	Modelo ingenuo subestima $\times 2.4$
6	Producto acotado $[0, 1]$	Soporte físico, muestra finita	GUM normal inválida en el borde

Las cuatro lecciones transversales que el estudiante debe extraer son:

1. **Un resultado sin incertidumbre es un resultado incompleto.** El valor nominal y la incertidumbre forman una unidad indivisible; reportar uno sin el otro es metodológicamente inaceptable a nivel doctoral.
2. **La LPU es una linealización; el Monte Carlo es el juez.** Cuando el modelo es no lineal, asimétrico, correlacionado o acotado, la GUM clásica puede fallar, y la validación numérica de (182) dictamina cuándo debe reportarse el resultado Monte Carlo.
3. **La estructura de la incertidumbre importa tanto como su magnitud.** Correlaciones (Ejemplos 1, 5), soporte acotado (Ejemplo 6) y no normalidad (Ejemplos 2, 3) cambian el resultado de forma cualitativa, no sólo cuantitativa.
4. **El presupuesto de incertidumbre es una herramienta de decisión.** Saber qué fuente domina la varianza convierte el análisis en una guía de inversión: dónde calibrar mejor, qué instrumento mejorar, qué tolerancia apretar.

Apéndice Semana 6. Referencias verificables

- A1. BIPM, IEC, IFCC, ILAC, ISO, IUPAC, IUPAP, OIML (2008). *Evaluation of measurement data — Guide to the expression of uncertainty in measurement (GUM)*. JCGM 100:2008. https://www.bipm.org/documents/20126/2071204/JCGM_100_2008_E.pdf
- A2. BIPM, IEC, IFCC, ILAC, ISO, IUPAC, IUPAP, OIML (2008). *Evaluation of measurement data — Supplement 1 to the GUM: Propagation of distributions using a Monte Carlo method*. JCGM 101:2008. https://www.bipm.org/documents/20126/2071204/JCGM_101_2008_E.pdf
- A3. Matsumoto, M. y Nishimura, T. (1998). Mersenne Twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation*, 8(1), 3–30. <https://doi.org/10.1145/272991.272995>
- A4. O'Neill, M. E. (2014). *PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation*. Technical Report HMC-CS-2014-0905, Harvey Mudd College. <https://www.cs.hmc.edu/tr/hmc-cs-2014-0905.pdf>
- A5. Harris, C. R. et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- A6. Efron, B. (1979). Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics*, 7(1), 1–26. <https://doi.org/10.1214/aos/1176344552>
- A7. Melchers, R. E. y Beck, A. T. (2018). *Structural Reliability Analysis and Prediction* (3.^a ed.). John Wiley & Sons. <https://doi.org/10.1002/9781119266105>

- A8. International Electrotechnical Commission (1988). *IEC 60949: Calculation of thermally permissible short-circuit currents, taking into account non-adiabatic heating effects*. IEC, Ginebra.

6.14. Anexo Semana 6. Códigos para Google Colab

Los seis códigos siguientes son autocontenidos y reproducibles: cada uno fija una semilla del generador PCG64 (`numpy.random.default_rng`), crea la carpeta Figuras/ y guarda la figura a 300 dpi. Todos los valores numéricos citados en el texto provienen de la ejecución de estos códigos. Para usarlos en Google Colab, basta copiarlos en una celda y ejecutarlos; la figura aparecerá en Figuras/ dentro del entorno.

6.14.1. Código 1 — Calibración de sensor: GUM vs Monte Carlo

Ajuste por mínimos cuadrados, predicción inversa, propagación analítica con correlación pendiente–intersección y validación Monte Carlo.

```

1      """
2      Semana 6 - Ejemplo 1: Calibracion de un sensor e
      incertidumbre del valor
3      medido (caso canonico GUM: modelo lineal, Tipo A + Tipo B,
      LPU vs Monte Carlo).
4      """
5      import numpy as np
6      import matplotlib
7      matplotlib.use('Agg')
8      import matplotlib.pyplot as plt
9      import matplotlib as mpl
10     from scipy import stats
11
12     mpl.rcParams.update({
13         'font.size': 10, 'axes.titlesize': 11, 'axes.labelsize':
14         10,
15         'figure.dpi': 110, 'savefig.dpi': 300, 'axes.grid': True
16     },
17     {'grid.alpha': 0.3, 'mathtext.fontset': 'cm',
18     })
19
20     rng = np.random.default_rng(20251117)
21     import os; os.makedirs('Figuras', exist_ok=True) # carpeta
      de salida en Colab
    
```

```

21     # ----- datos de calibracion (sinteticos, reproducibles)
-----
22     a_true, b_true, sigma_y = 2.00, 0.500, 0.15      # mV, mV/
kPa, mV
23     x_cal = np.linspace(0.0, 100.0, 11)             # patron de
referencia [kPa]
24     y_cal = a_true + b_true*x_cal + rng.normal(0, sigma_y, x_cal
.size)
25
26     n = x_cal.size
27     Xmat = np.column_stack([np.ones(n), x_cal])
28     beta_hat, *_ = np.linalg.lstsq(Xmat, y_cal, rcond=None)
29     a_hat, b_hat = beta_hat
30     resid = y_cal - Xmat @ beta_hat
31     dof = n - 2
32     s2 = resid @ resid / dof                        # varianza
residual (Tipo A)
33     cov_beta = s2 * np.linalg.inv(Xmat.T @ Xmat)    # cov(a,b)
34     ua, ub = np.sqrt(np.diag(cov_beta))
35     rho_ab = cov_beta[0, 1] / (ua*ub)
36
37     # ----- medicion nueva: lectura y0 -> medir x0 -----
38     y0 = 27.00                                     # mV
lectura del sensor
39     res_sensor = 0.01                              # mV
resolucion (rectangular)
40     u_res = res_sensor / np.sqrt(12)
41     u_rep = 0.05                                    # mV
repetibilidad (Tipo A, 1 lectura)
42     u_y0 = np.sqrt(u_res**2 + u_rep**2)
43     half_ref = 0.20                                # kPa
patron (rectangular, certificado)
44     u_ref = half_ref / np.sqrt(3)
45
46     x0 = (y0 - a_hat) / b_hat
47
48     # ----- GUM (LPU) con coeficientes de sensibilidad
-----
49     c_y0 = 1.0 / b_hat
50     c_a = -1.0 / b_hat
51     c_b = -(y0 - a_hat) / b_hat**2
52     c_ref = 1.0

```

```

53     # termino de regresion con covarianza (a,b correlacionados)
54     var_reg = np.array([c_a, c_b]) @ cov_beta @ np.array([c_a,
55     c_b])
56     u_c2 = c_y0**2 * u_y0**2 + var_reg + c_ref**2 * u_ref**2
57     u_c = np.sqrt(u_c2)
58
59     # Welch-Satterthwaite (Tipo A: dof; Tipo B: dof infinito)
60     comp = {
61         'regresion': (np.sqrt(var_reg), dof),
62         'lectura_y0_rep': (abs(c_y0)*u_rep, 9),          # ~10
63         'lecturas de repetibilidad'
64         'lectura_y0_res': (abs(c_y0)*u_res, np.inf),
65         'patron_ref': (u_ref, np.inf),
66     }
67     num = u_c**4
68     den = sum((ui**4 / vi) for (ui, vi) in comp.values() if np.
69     isfinite(vi))
70     nu_eff = num / den if den > 0 else np.inf
71     k = stats.t.ppf(0.975, nu_eff) if np.isfinite(nu_eff) else
72     1.96
73     U_gum = k * u_c
74
75     # ----- Monte Carlo (JCGM 101) -----
76     M = 200_000
77     ab_s = rng.multivariate_normal([a_hat, b_hat], cov_beta,
78     size=M)
79     y0_s = y0 + rng.normal(0, u_y0, M)
80     ref_s = rng.uniform(-half_ref, half_ref, M)          #
81     desplazamiento de escala del patron
82     x0_s = (y0_s - ab_s[:, 0]) / ab_s[:, 1] + ref_s
83     x0_mc = x0_s.mean()
84     u_mc = x0_s.std(ddof=1)
85     lo_mc, hi_mc = np.percentile(x0_s, [2.5, 97.5])
86
87     # ----- validacion GUM por MC (JCGM 101, tolerancia
88     numerica) -----
89     ndig = 2          # cifras
90     significativas de u_c
91     delta = 0.5 * 10**(np.floor(np.log10(u_c)) - ndig + 1)
92     lo_gum, hi_gum = x0 - U_gum, x0 + U_gum
93     dlow = abs(lo_gum - lo_mc); dhigh = abs(hi_gum - hi_mc)
94     validado = (dlow <= delta) and (dhigh <= delta)

```

```

87
88     print("=== EJEMPLO 1: CALIBRACION ===")
89     print(f"Ajuste: a={a_hat:.4f} mV (u={ua:.4f}), b={b_hat:.5f}
mV/kPa (u={ub:.5f}), rho_ab={rho_ab:+.3f}")
90     print(f"s (residual)={np.sqrt(s2):.4f} mV, dof={dof}")
91     print(f"x0 (medido) = {x0:.3f} kPa")
92     print(f"u_y0={u_y0:.4f} mV, u_ref={u_ref:.4f} kPa")
93     print(f"[GUM] u_c={u_c:.4f} kPa | nu_eff={nu_eff:.1f} | k={k
:.3f} | U={U_gum:.4f} kPa")
94     print(f"          intervalo 95%: [{lo_gum:.3f}, {hi_gum:.3f}]
kPa")
95     print(f"[MC ] media={x0_mc:.3f}, u={u_mc:.4f} kPa | 95%: [{
lo_mc:.3f}, {hi_mc:.3f}] kPa")
96     print(f"tol delta={delta:.4f} | d_low={dlow:.4f} d_high={
dhigh:.4f} -> validado={validado}")
97     budget = {kk: (vv[0], 100*vv[0]**2/u_c2) for kk, vv in comp.
items()}
98     print("Presupuesto (u_i [kPa], % var):")
99     for kk, (ui, pc) in budget.items():
100     print(f"    {kk:16s} {ui:.4f} {pc:5.1f}%")
101
102     # ----- figura -----
103     fig, axs = plt.subplots(1, 4, figsize=(16, 3.7))
104
105     ax = axs[0]
106     xx = np.linspace(-5, 105, 200)
107     yhat = a_hat + b_hat*xx
108     # banda de prediccion 95%
109     Xp = np.column_stack([np.ones_like(xx), xx])
110     var_pred = s2*(1 + np.einsum('ij,jk,ik->i', Xp, np.linalg.
inv(Xmat.T@Xmat), Xp))
111     tval = stats.t.ppf(0.975, dof)
112     band = tval*np.sqrt(var_pred)
113     ax.fill_between(xx, yhat-band, yhat+band, color='#c7d9ef',
label='banda 95%')
114     ax.plot(xx, yhat, '-', color='#0b3d91', lw=1.8, label='
ajuste OLS')
115     ax.plot(x_cal, y_cal, 'o', color='#b22222', ms=5, label='
puntos de calibracion')
116     ax.axhline(y0, color='gray', ls=':', lw=1)
117     ax.plot([x0], [y0], 'D', color='k', ms=6)
118     ax.annotate(f'$y_0={y0}$ mV\n$x_0={x0:.2f}$ kPa', (x0, y0),

```



```

119     xytext=(x0-46, y0+1.5), fontsize=8,
120     arrowprops=dict(arrowstyle='->', lw=0.8))
121     ax.set_xlabel('referencia $x$ [kPa]'); ax.set_ylabel('salida
    $y$ [mV]')
122     ax.set_title('(a) Curva de calibracion'); ax.legend(fontsize
    =7.5, loc='upper left')
123
124     ax = axs[1]
125     ax.stem(x_cal, resid*1e3, basefmt=' ')
126     ax.axhline(0, color='k', lw=0.8)
127     ax.axhline(np.sqrt(s2)*1e3, color='r', ls='--', lw=0.9,
    label=r'$\pm s$')
128     ax.axhline(-np.sqrt(s2)*1e3, color='r', ls='--', lw=0.9)
129     ax.set_xlabel('referencia $x$ [kPa]'); ax.set_ylabel(r'
    residual [$\mu V$]')
130     ax.set_title('(b) Residuales del ajuste'); ax.legend(
    fontsize=8)
131
132     ax = axs[2]
133     ax.hist(x0_s, bins=120, density=True, color='#9ecae1',
    edgecolor='none', label='MC')
134     xg = np.linspace(x0-4*u_c, x0+4*u_c, 300)
135     ax.plot(xg, stats.norm.pdf(xg, x0, u_c), 'r-', lw=1.6, label
    ='GUM (normal)')
136
137     ax.axvline(lo_mc, color='k', ls='--', lw=1); ax.axvline(
    hi_mc, color='k', ls='--', lw=1)
138     ax.axvline(lo_gum, color='r', ls=':', lw=1.2); ax.axvline(
    hi_gum, color='r', ls=':', lw=1.2)
139     ax.set_xlabel('$x_0$ [kPa]'); ax.set_ylabel('densidad')
140     ax.set_title('(c) GUM vs Monte Carlo'); ax.legend(fontsize
    =8)
141
142     ax = axs[3]
143     Ms = np.unique(np.logspace(2, np.log10(M), 40).astype(int))
144     us = [x0_s[:m].std(ddof=1) for m in Ms]
145     ax.semilogx(Ms, us, '-', color='#0b3d91', lw=1.5)
146     ax.axhline(u_c, color='r', ls='--', lw=1.2, label=f'$u_c$
    GUM = {u_c:.3f}')
147     ax.fill_between(Ms, u_c*(1-1/np.sqrt(2*np.array(Ms))),
    u_c*(1+1/np.sqrt(2*np.array(Ms))), color='#fdd', alpha=0.5)
148     ax.set_xlabel('numero de pruebas MC, $M$'); ax.set_ylabel('
    $u(x_0)$ [kPa]')

```

```

150     ax.set_title('(d) Convergencia de Monte Carlo'); ax.legend(
        fontsize=8)
151
152     fig.suptitle('Calibracion de sensor: incertidumbre del valor
        medido (GUM y Monte Carlo)', y=1.03)
153     plt.savefig('Figuras/fig_s6_ej1_calibracion.png', dpi=300,
        bbox_inches='tight')
154     # plt.show()
155     print("Figura guardada.")

```

Listing 37: Semana 6 – Código 1: Calibración de sensor: GUM vs Monte Carlo. Ejecutar en Google Colab.

6.14.2. Código 2 — Cortocircuito en cable OPGW: no linealidad y probabilidad de falla

Modelo adiabático IEC 60949, propagación de una exponencial fuertemente no lineal y estimación de la probabilidad de exceder el límite térmico.

```

1      """
2      Semana 6 - Ejemplo 2: Elevacion adiabatica de temperatura de
        un conductor
3      OPGW ante cortocircuito (modelo exponencial fuertemente no
        lineal).
4      Modelo IEC 60949 adiabatico:  $I^2 t = K^2 S^2 \ln((T_f + \beta) / (T_i + \beta))$ 
        =>  $T_f = (T_i + \beta) \exp(I^2 t / (K^2 S^2)) - \beta$ 
5      """
6
7      import numpy as np
8      import matplotlib
9      matplotlib.use('Agg')
10     import matplotlib.pyplot as plt
11     import matplotlib as mpl
12     from scipy import stats
13
14     mpl.rcParams.update({
15         'font.size': 10, 'axes.titlesize': 11, 'axes.labelsize':
        10,
16         'figure.dpi': 110, 'savefig.dpi': 300, 'axes.grid': True
        ,
17         'grid.alpha': 0.3, 'mathtext.fontset': 'cm',
18     })
19     rng = np.random.default_rng(60606)

```

```

20     import os; os.makedirs('Figuras', exist_ok=True) # carpeta
    de salida en Colab
21
22     def Tf(I, t, S, Ti, K, beta):
23     return (Ti + beta)*np.exp(I**2 * t / (K**2 * S**2)) - beta
24
25     # ----- magnitudes de entrada (valor, u, distribucion)
    -----
26     # I [A], t [s], S [mm2], Ti [C], K [A s0.5 / mm2], beta [
    C] (aluminio)
27     IO, uI = 12.5e3, 0.04*12.5e3 # Normal (4% rel: CT +
    calculo de falla)
28     t0, ht = 0.50, 0.06 # rectangular +-0.06 s
29     S0, hS = 90.0, 0.02*90.0 # rectangular +-2% (
    tolerancia de fabricacion)
30     Ti0, hTi = 40.0, 15.0 # rectangular +-15 C (
    condicion de operacion)
31     K0, uK = 148.0, 1.5 # Normal (~1%
    propiedad de material)
32     beta0, ubeta = 228.0, 3.0 # Normal
33     T_lim = 200.0 # C limite de
    recocido del aluminio
34
35     y0 = Tf(IO, t0, S0, Ti0, K0, beta0)
36
37     # ----- GUM (LPU): sensibilidades analiticas (diferencias
    centradas) -----
38     def num_partial(f, args, i, h):
39     a = list(args); b = list(args)
40     a[i] += h; b[i] -= h
41     return (f(*a) - f(*b)) / (2*h)
42
43     args = [IO, t0, S0, Ti0, K0, beta0]
44     us = [uI, ht/np.sqrt(3), hS/np.sqrt(3), hTi/np.sqrt(3), uK,
    ubeta]
45     names = ['I', 't', 'S', 'Ti', 'K', 'beta']
46     steps = [1.0, 1e-4, 1e-3, 1e-3, 1e-2, 1e-2]
47     c = [num_partial(Tf, args, i, steps[i]) for i in range(6)]
48     contrib = [(c[i]*us[i])**2 for i in range(6)]
49     u_c = np.sqrt(sum(contrib))
50     U_gum = 2.0*u_c
51     lo_gum, hi_gum = y0 - U_gum, y0 + U_gum
    
```

```

52  # Pf gaussiano (aprox GUM)
53  Pf_gum = 1 - stats.norm.cdf(T_lim, loc=y0, scale=u_c)
54
55  # ----- Monte Carlo (JCGM 101) -----
56  M = 300_000
57  Is = rng.normal(I0, uI, M)
58  ts = rng.uniform(t0-ht, t0+ht, M)
59  Ss = rng.uniform(S0-hS, S0+hS, M)
60  Tis = rng.uniform(Ti0-hTi, Ti0+hTi, M)
61  Ks = rng.normal(K0, uK, M)
62  bs = rng.normal(beta0, ubeta, M)
63  Y = Tf(Is, ts, Ss, Tis, Ks, bs)
64  y_mc = Y.mean(); u_mc = Y.std(ddof=1)
65  lo_mc, hi_mc = np.percentile(Y, [2.5, 97.5])
66  g1 = stats.skew(Y)
67  Pf_mc = np.mean(Y > T_lim)
68  # error estandar de Pf
69  se_Pf = np.sqrt(Pf_mc*(1-Pf_mc)/M)
70
71  # ----- validacion GUM por MC -----
72  ndig = 2
73  delta = 0.5*10**(np.floor(np.log10(u_c)) - ndig + 1)
74  dlow = abs(lo_gum-lo_mc); dhigh = abs(hi_gum-hi_mc)
75  validado = (dlow <= delta) and (dhigh <= delta)
76
77  print("=== EJEMPLO 2: OPGW CORTOCIRCUITO ===")
78  print(f"Tf nominal = {y0:.1f} C")
79  print(f"[GUM] u_c={u_c:.2f} C | U={U_gum:.2f} | 95%: [{lo_gum:.1f}, {hi_gum:.1f}] | Pf={Pf_gum*100:.2f}%")
80  print(f"[MC ] media={y_mc:.1f} C | u={u_mc:.2f} C | 95%: [{lo_mc:.1f}, {hi_mc:.1f}]")
81  print(f"      skew={g1:+.3f} | Pf(Tf>{T_lim:.0f})={Pf_mc*100:.2f}% (+-{se_Pf*100:.2f}%)" )
82  print(f"      sesgo media MC-nominal = {y_mc-y0:+.2f} C")
83  print(f"tol delta={delta:.3f} | d_low={dlow:.2f} d_high={dhigh:.2f} -> validado={validado}")
84  print("Presupuesto GUM (u_i*c_i [C], % var):")
85  order = np.argsort(contrib)[::-1]
86  for i in order:
87  print(f"      {names[i]:5s} c={c[i]:+.4g} u={us[i]:.4g} | c*u
88  |={abs(c[i]*us[i]):7.2f} {100*contrib[i]/u_c**2:5.1f}%")

```

```

89     # ----- figura -----
90     fig, axs = plt.subplots(1, 4, figsize=(16, 3.7))
91
92     ax = axs[0]
93     arg = np.linspace(0, 1.0, 200)
94     ax.plot(arg, (Ti0+beta0)*np.exp(arg)-beta0, '-', color='#0
b3d91', lw=1.8)
95     arg0 = I0**2*t0/(K0**2*S0**2)
96     ax.plot([arg0], [y0], 'D', color='b', ms=7, label='punto de
operacion')
97     # propagar +-u en el argumento para ilustrar no linealidad
98     da = np.std(Is**2*ts/(Ks**2*Ss**2))
99     ax.axvspan(arg0-da, arg0+da, color='#cfe2f3', alpha=0.6,
label=r'$\pm u$ en el argumento')
100    ax.axhline(T_lim, color='r', ls='--', lw=1, label='limite
200 C')
101    ax.set_xlabel(r'argumento  $I^2 t / (K^2 S^2)$ '); ax.set_ylabel
('$T_f$ [C]')
102    ax.set_title('(a) No linealidad exponencial'); ax.legend(
fontsize=7.5, loc='upper left')
103
104    ax = axs[1]
105    ax.hist(Y, bins=140, density=True, color='#9ecae1',
edgecolor='none', label='MC')
106    xg = np.linspace(y0-4*u_c, y0+4*u_c, 300)
107    ax.plot(xg, stats.norm.pdf(xg, y0, u_c), 'r-', lw=1.6, label
='GUM (normal)')
108    ax.axvline(T_lim, color='k', ls='--', lw=1.3, label='$T_{lim
}$')
109    ax.fill_between(xg[xg>=T_lim], 0, stats.norm.pdf(xg[xg>=
T_lim], y0, u_c),
110    color='red', alpha=0.25)
111    ax.set_xlabel('$T_f$ [C]'); ax.set_ylabel('densidad')
112    ax.set_title(f'(b) Distribucion (sesgo  $\gamma_1 = \{g1 : +.2f\}$ 
)'); ax.legend(fontsize=7.5)
113
114    ax = axs[2]
115    pct = np.array([100*ci/u_c**2 for ci in contrib])
116    idx = np.argsort(pct)
117    ax.barh(np.array(names)[idx], pct[idx], color='#0b3d91')
118    ax.set_xlabel('% de la varianza combinada')
119    ax.set_title('(c) Presupuesto de incertidumbre')

```

```

120     for yi, p in zip(range(len(idx)), pct[idx]):
121         ax.text(p+0.8, yi, f'{p:.1f}%', va='center', fontsize=8)
122         ax.set_xlim(0, max(pct)*1.18)
123
124         ax = axs[3]
125         qs = np.linspace(0, 100, 200)
126         ax.plot(np.percentile(Y, qs), qs/100, '-', color='#0b3d91',
127                 lw=1.8, label='MC (empirica)')
128         ax.plot(stats.norm.ppf(np.clip(qs/100, 1e-4, 1-1e-4), y0, u_c),
129                 qs/100, 'r--', lw=1.4, label='GUM (normal)')
130         ax.axhline(0.025, color='gray', ls=':', lw=0.8); ax.axhline(
131             0.975, color='gray', ls=':', lw=0.8)
132         ax.axvline(T_lim, color='k', ls='--', lw=1)
133         ax.set_xlabel('$T_f$ [C]'); ax.set_ylabel('probabilidad
134             acumulada')
135         ax.set_title('(d) CDF: cobertura GUM vs MC'); ax.legend(
136             fontsize=8, loc='lower right')
137
138     fig.suptitle('OPGW ante cortocircuito: propagacion no lineal
139         de incertidumbre y probabilidad de exceder el limite termico
140         ', y=1.03)
141     plt.savefig('Figuras/fig_s6_ej2_opgw_cortocircuito.png', dpi
142                 =300, bbox_inches='tight')
143     # plt.show()
144     print("Figura guardada.")
    
```

Listing 38: Semana 6 – Código 2: Cortocircuito en cable OPGW: no linealidad y probabilidad de falla. Ejecutar en Google Colab.

6.14.3. Código 3 — Confiabilidad estructural: losa maciza vs reticular

Deflexión por servicio, índice de confiabilidad β por Monte Carlo y por FORM, y reparto de varianza.

```

1     """
2     Semana 6 - Ejemplo 3: Confiabilidad estructural de una losa
3     (solida vs
4     reticular/waffle). Estado limite de servicio por deflexion:
5     g = delta_adm - delta,    delta = alpha w L^4 / (E I),
6     delta_adm = L/360
7     Se calcula la probabilidad de falla Pf=P(g<0) y el indice de
8     confiabilidad
    
```

```

6     beta, y se propaga la incertidumbre de delta (GUM vs Monte
    Carlo).
7     """
8     import numpy as np
9     import matplotlib
10    matplotlib.use('Agg')
11    import matplotlib.pyplot as plt
12    import matplotlib as mpl
13    from scipy import stats
14
15    mpl.rcParams.update({
16        'font.size': 10, 'axes.titlesize': 11, 'axes.labelsize':
17        10,
18        'figure.dpi': 110, 'savefig.dpi': 300, 'axes.grid': True
19    },
20        'grid.alpha': 0.3, 'mathtext.fontset': 'cm',
21    })
22    rng = np.random.default_rng(360360)
23    import os; os.makedirs('Figuras', exist_ok=True) # carpeta
    de salida en Colab
24
25    alpha = 5.0/384.0 # coeficiente de deflexion (
26    apoyo simple, franja 1 m)
27    L0, hL = 6.0, 0.02 # m, rectangular +-0.02 m
28
29    # --- propiedades por tipo de losa (franja de 1 m de ancho)
30    ---
31    # Solida: mas pesada, menor I. Reticular: mas ligera, mayor
32    I (mas peralte).
33    designs = {
34        'Solida': dict(wd=5.2, cov_wd=0.10, I=4.30e-4, cov_I
35        =0.12),
36        'Reticular': dict(wd=3.6, cov_wd=0.10, I=5.00e-4, cov_I
37        =0.15),
38    }
39    wl0, cov_wl = 2.0, 0.25 # kPa carga viva (Normal trunc
40    .), COV alto
41    fc0, cov_fc = 25.0, 0.12 # MPa, lognormal
42    M = 400_000
43
44    def E_of_fc(fc): # ACI: E = 4700 sqrt(fc) [MPa]
45        -> Pa

```

```

37     return 4700.0*np.sqrt(fc)*1e6
38
39     results = {}
40     fig, axs = plt.subplots(1, 4, figsize=(16, 3.7))
41     colors = {'Solida': '#b22222', 'Reticular': '#0b3d91'}
42
43     # muestras compartidas de variables comunes
44     L_s = rng.uniform(L0-hL, L0+hL, M)
45     fc_s = np.exp(rng.normal(np.log(fc0) - 0.5*np.log(1+cov_fc
46 **2),
47 np.sqrt(np.log(1+cov_fc**2)), M)) # lognormal
48     E_s = E_of_fc(fc_s)
49     wl_s = np.clip(rng.normal(wl0, cov_wl*wl0, M), 0, None)
50
51     for name, d in designs.items():
52         wd_s = rng.normal(d['wd'], d['cov_wd']*d['wd'], M)
53         I_s = np.clip(rng.normal(d['I'], d['cov_I']*d['I'], M), 1e
-6, None)
54         w_s = (wd_s + wl_s)*1e3 # kPa -> N/m (
franja 1 m)
55         delta_s = alpha * w_s * L_s**4 / (E_s * I_s) # m
56         delta_adm = L_s/360.0
57         g = delta_adm - delta_s
58         Pf = np.mean(g < 0)
59         se = np.sqrt(Pf*(1-Pf)/M)
60         beta = -stats.norm.ppf(Pf) if Pf > 0 else np.inf
61         beta_form = g.mean()/g.std(ddof=1) # indice de
Cornell (FORM 1er orden)
62         # nominal y GUM (LPU) sobre delta
63         w0 = (d['wd']+wl0)*1e3; E0 = E_of_fc(fc0); I0 = d['I']
64         delta0 = alpha*w0*L0**4/(E0*I0)
65         # sensibilidades relativas (modelo producto/potencia)
66         rel = np.sqrt((cov_wl*wl0/(d['wd']+wl0))**2 + (d['cov_wd']*d
['wd']/(d['wd']+wl0))**2
67 + (4*hL/np.sqrt(3)/L0)**2 + (0.5*cov_fc)**2 + d['cov_I']**2)
68         u_delta_gum = rel*delta0
69         results[name] = dict(delta_s=delta_s, delta0=delta0, u_gum=
u_delta_gum,
70 u_mc=delta_s.std(ddof=1), Pf=Pf, se=se, beta=beta,
71 beta_form=beta_form, delta_adm0=L0/360, g=g)
72     print(f"=== {name} ===")
73     print(f"    delta0={delta0*1e3:.2f} mm | delta_adm={L0/360*1e3

```



```

        :.2f} mm | margen={delta0/(L0/360)*100:.1f}% de adm")
73     print(f"   u_delta: GUM={u_delta_gum*1e3:.3f} mm | MC={
delta_s.std(ddof=1)*1e3:.3f} mm")
74     print(f"   Pf={Pf*100:.3f}% (+-{se*100:.3f}%) | beta_MC={beta
:.3f} | beta_FORM={beta_form:.3f}")

75
76     # (a) histograma de delta para ambas losas
77     ax = axs[0]
78     for name in designs:
79         r = results[name]
80         ax.hist(r['delta_s']*1e3, bins=120, density=True, alpha
=0.55,
81         color=colors[name], edgecolor='none', label=name)
82         ax.axvline(L0/360*1e3, color='k', ls='--', lw=1.4, label=r'$
\delta_{adm}=L/360$')
83         ax.set_xlabel(r'deflexion $\delta$ [mm]'); ax.set_ylabel('
densidad')
84         ax.set_title('(a) Distribucion de la deflexion'); ax.legend(
fontsize=8)
85         ax.set_xlim(0, max(results['Solida']['delta_s'].max(), L0
/360*1.4)*1e3)

86
87     # (b) Pf y beta comparados
88     ax = axs[1]
89     nm = list(designs.keys())
90     Pfs = [results[n]['Pf']*100 for n in nm]
91     betas = [results[n]['beta'] for n in nm]
92     bars = ax.bar(nm, Pfs, color=[colors[n] for n in nm], alpha
=0.8)
93     ax.set_ylabel(r'$P_f$ [%]'); ax.set_title(r'(b) Probabilidad
de falla e indice $\beta$')
94     for b, n in zip(bars, nm):
95         bval = results[n]['beta'] if np.isfinite(results[n]['beta'])
else results[n]['beta_form']
96         ax.text(b.get_x()+b.get_width()/2, b.get_height()*1.02 + 1e
-3,
97         f'$P_f$={results[n]["Pf"]*100:.2f}%\n$\beta$={bval:.2f}',
98         ha='center', fontsize=8)
99         ax.set_ylim(0, max(Pfs)*1.5 + 0.1)

100
101     # (c) contribuciones de varianza (Reticular) por re-muestreo
congelando variables
    
```

```

102     ax = axs[2]
103     base = results['Reticular']['delta_s'].var()
104     labels_v = ['carga viva', 'carga muerta', 'span L', "f'c (E)",
105               'inercia I']
106     # estimacion por correlacion-cuadrada (aprox first-order
107     indices)
108     d = designs['Reticular']
109     wd_s = rng.normal(d['wd'], d['cov_wd']*d['wd'], M)
110     I_s = np.clip(rng.normal(d['I'], d['cov_I']*d['I'], M), 1e
111     -6, None)
112     w_s = (wd_s + wl_s)*1e3
113     delta_full = alpha*w_s*L_s**4/(E_s*I_s)
114     def s1(var_samp, ref):
115         # indice de sensibilidad de primer orden (Var de E[Y/Xi])
116         aprox por binning
117         bins = np.quantile(var_samp, np.linspace(0,1,31))
118         idx = np.clip(np.digitize(var_samp, bins[1:-1]), 0, 29)
119         means = np.array([ref[idx==k].mean() if np.any(idx==k) else
120         np.nan for k in range(30)])
121         return np.nanvar(means)/ref.var()
122     S1 = [s1(wl_s, delta_full), s1(wd_s, delta_full), s1(L_s,
123     delta_full),
124     s1(fc_s, delta_full), s1(I_s, delta_full)]
125     S1 = np.array(S1); S1 = S1/ S1.sum()*100
126     order = np.argsort(S1)
127     ax.barh(np.array(labels_v)[order], S1[order], color='#0b3d91
128     ')
129     for yi, p in zip(range(len(order)), S1[order]):
130         ax.text(p+0.6, yi, f'{p:.0f}%', va='center', fontsize=8)
131         ax.set_xlabel('contribucion a la varianza [%]')
132         ax.set_title('(c) Sensibilidad (losa reticular)'); ax.
133         set_xlim(0, max(S1)*1.2)
134
135     # (d) convergencia de Pf
136     ax = axs[3]
137     Ms = np.unique(np.logspace(2.5, np.log10(M), 40).astype(int)
138     )
139     for name in designs:
140         g = results[name]['g']
141         pfs = [np.mean(g[:m] < 0) for m in Ms]
142         ax.semilogx(Ms, np.array(pfs)*100, '-', color=colors[name],
143         lw=1.5, label=name)

```

```

134     ax.axhline(results[name]['Pf']*100, color=colors[name], ls='
: ', lw=1)
135     ax.set_xlabel('numero de pruebas MC, $M$'); ax.set_ylabel(r'
$P_f$ [%]')
136     ax.set_title('(d) Convergencia de $P_f$'); ax.legend(
fontsize=8)
137
138     fig.suptitle('Confiabilidad de losa solida vs reticular:
deflexion de servicio, $P_f$ e indice $\beta$', y=1.03)
139     plt.savefig('Figuras/fig_s6_ej3_losa_confiabilidad.png', dpi
=300, bbox_inches='tight')
140     # plt.show()
141     print("Figura guardada.")

```

Listing 39: Semana 6 – Código 3: Confiabilidad estructural: losa maciza vs reticular. Ejecutar en Google Colab.

6.14.4. Código 4 — Optimización de bomba centrífuga: modelo multiplicativo y BEP

Eficiencia hidráulica multiplicativa, propagación de la incertidumbre al punto de mejor eficiencia mediante ajuste parabólico.

```

1     """
2     Semana 6 - Ejemplo 4: Eficiencia de una bomba centrifuga (
3     modelo
4     multiplicativo) e incertidumbre del punto de mejor
5     eficiencia (BEP).
6     eta = rho g Q H / P      (modelo producto/cociente ->
7     cuadratura relativa)
8     Se propaga la incertidumbre por punto (GUM) y se localiza el
9     BEP por ajuste
10    parabolico, propagando la incertidumbre de (Q_BEP, eta_max)
11    por Monte Carlo.
12    """
13
14    import numpy as np
15    import matplotlib
16    matplotlib.use('Agg')
17    import matplotlib.pyplot as plt
18    import matplotlib as mpl
19    from scipy import stats
20
21    mpl.rcParams.update({

```

```

16         'font.size': 10, 'axes.titlesize': 11, 'axes.labelsize':
17         10,
18         'figure.dpi': 110, 'savefig.dpi': 300, 'axes.grid': True
19     ,
20     'grid.alpha': 0.3, 'mathtext.fontset': 'cm',
21     })
22     rng = np.random.default_rng(404404)
23     import os; os.makedirs('Figuras', exist_ok=True) # carpeta
24     de salida en Colab
25
26     g = 9.81
27     rho0, u_rho = 997.0, 0.3 # kg/m3 (agua ~25 C
28     ), u Tipo B
29     # incertidumbres relativas de instrumentos
30     relQ, relH, relP = 0.020, 0.015, 0.015 # caudalimetro,
31     head, potencia al eje
32
33     # curva "verdadera" de eficiencia (para generar datos de
34     ensayo)
35     Q_test = np.array([20, 30, 40, 50, 60, 70, 80])*1e-3/60 #
36     m3/s (L/min->m3/s)
37     Q_bep_true, eta_max_true, curv = 50e-3/60, 0.78, 1.0e6
38     eta_true = eta_max_true - curv*(Q_test - Q_bep_true)**2
39     H_true = 30.0 - 1.5e3*(Q_test - 30e-3/60)**2 - 8e3*Q_test
40     # m, curva H-Q decreciente
41     P_true = rho0*g*Q_test*H_true/eta_true
42     # W
43
44     # datos medidos (una corrida, ruido de instrumento)
45     Qm = Q_test*(1 + rng.normal(0, relQ, Q_test.size))
46     Hm = H_true*(1 + rng.normal(0, relH, Q_test.size))
47     Pm = P_true*(1 + rng.normal(0, relP, Q_test.size))
48     eta_m = rho0*g*Qm*Hm/Pm
49
50     # GUM por punto: modelo multiplicativo -> (u_eta/eta)^2 =
51     sum (rel_i)^2
52     rel_eta = np.sqrt((u_rho/rho0)**2 + relQ**2 + relH**2 + relP
53     **2)
54     u_eta_pt = rel_eta*eta_m
55     print("=== EJEMPLO 4: BOMBA CENTRIFUGA ===")
56     print(f"u_eta/eta (GUM, multiplicativo) = {rel_eta*100:.3f}%")
57     ")

```

```

46     print("Contribuciones relativas a la varianza:")
47     for nm, r in [('rho', u_rho/rho0), ('Q', relQ), ('H', relH),
48                  ('P', relP)]:
49         print(f"    {nm:4s} rel={r*100:.3f}%    ({100*r**2/rel_eta
50             **2:.1f}% de la var.)")
51
52     # --- BEP por ajuste parabolico de los puntos medidos ---
53     def fit_bep(Q, eta):
54         A = np.column_stack([Q**2, Q, np.ones_like(Q)])
55         a, b, c = np.linalg.lstsq(A, eta, rcond=None)[0]
56         Qb = -b/(2*a)
57         eb = a*Qb**2 + b*Qb + c
58         return Qb, eb
59
60     Qb0, eb0 = fit_bep(Qm, eta_m)
61     print(f"BEP medido: Q_BEP={Qb0*6e4:.2f} L/min, eta_max={eb0
62         *100:.2f}%")
63
64     # --- Monte Carlo: cada punto medido fluctua segun su
65     incertidumbre ---
66     M = 100_000
67     Qbep_s = np.empty(M); etam_s = np.empty(M)
68     for k in range(M):
69         rho_k = rng.normal(rho0, u_rho)
70         Qk = Qm*(1 + rng.normal(0, relQ, Qm.size))
71         Hk = Hm*(1 + rng.normal(0, relH, Hm.size))
72         Pk = Pm*(1 + rng.normal(0, relP, Pm.size))
73         eta_k = rho_k*g*Qk*Hk/Pk
74         Qbep_s[k], etam_s[k] = fit_bep(Qk, eta_k)
75     Qbep_mc = Qbep_s.mean()*6e4; uQbep = Qbep_s.std(ddof=1)*6e4
76     etam_mc = etam_s.mean()*100; uetam = etam_s.std(ddof=1)*100
77     print(f"[MC] Q_BEP = {Qbep_mc:.2f} +- {uQbep:.2f} L/min    (k
78         =2: +-{2*uQbep:.2f})")
79     print(f"[MC] eta_max = {etam_mc:.2f} +- {uetam:.2f} %    (k
80         =2: +-{2*uetam:.2f})")
81     rho_corr = np.corrcoef(Qbep_s, etam_s)[0,1]
82     print(f"correlacion(Q_BEP, eta_max) = {rho_corr:+.3f}")
83
84     # ----- figura -----
85     fig, axs = plt.subplots(1, 4, figsize=(16, 3.7))
86
87     ax = axs[0]
88     Qg = np.linspace(Q_test.min(), Q_test.max(), 200)

```

```

82     A = np.column_stack([Qm**2, Qm, np.ones_like(Qm)])
83     coef = np.linalg.lstsq(A, eta_m, rcond=None)[0]
84     etafit = coef[0]*Qg**2 + coef[1]*Qg + coef[2]
85     # banda de incertidumbre de la curva por MC
86     band = np.std([ (lambda cc: cc[0]*Qg**2+cc[1]*Qg+cc[2])(
87         np.linalg.lstsq(np.column_stack([ (Qm*(1+rng.normal(0,relQ,
88         Qm.size)))*2,
89         Qm*(1+rng.normal(0,relQ,Qm.size))), np.ones_like(Qm)]),
90         eta_m*(1+rng.normal(0,rel_eta,Qm.size))), rcond=None)[0])
91     for _ in range(400)], axis=0)
92     ax.fill_between(Qg*6e4, (etafit-2*band)*100, (etafit+2*band)
93     *100,
94     color='#cfe2f3', label='banda 95%')
95     ax.plot(Qg*6e4, etafit*100, '-', color='#0b3d91', lw=1.8,
96     label='ajuste parabolico')
97     ax.errorbar(Qm*6e4, eta_m*100, yerr=2*u_eta_pt*100, fmt='o',
98     color='#b22222',
99     ms=5, capsize=3, lw=1, label='puntos $\pm$ U$ (k=2)')
100     ax.axvline(Qbep_mc, color='g', ls='--', lw=1.2, label='$Q_{BEP}$')
101     ax.set_xlabel('caudal $Q$ [L/min]'); ax.set_ylabel(r'
102     eficiencia $\eta$ [%]')
103     ax.set_title('(a) Curva $\eta$-$Q$ con incertidumbre'); ax.
104     legend(fontsize=7.5, loc='lower center')
105
106     ax = axs[1]
107     labels = [r'$\rho$', '$Q$', '$H$', '$P$']
108     rels = np.array([u_rho/rho0, relQ, relH, relP])
109     pc = 100*rels**2/rel_eta**2
110     ax.bar(labels, pc, color='#0b3d91')
111     for i, p in enumerate(pc):
112         ax.text(i, p+1, f'{p:.0f}%', ha='center', fontsize=8)
113     ax.set_ylabel('% de la varianza de $\eta$')
114     ax.set_title(f'(b) Presupuesto: $u_{\eta}/\eta$={rel_eta
115     *100:.2f}%'); ax.set_ylim(0, max(pc)*1.2)
116
117     ax = axs[2]
118     ax.hist(etam_s*100, bins=100, density=True, color='#9ecae1',
119     edgecolor='none', label='MC')
120     xg = np.linspace(etam_mc-4*uetam, etam_mc+4*uetam, 200)
121     ax.plot(xg, stats.norm.pdf(xg, etam_mc, uetam), 'r-', lw
122     =1.6, label='GUM (normal)')

```

```

114     ax.set_xlabel(r'$\eta_{\max}$ [%]'); ax.set_ylabel('densidad'
115 )
116     ax.set_title('(c) Incertidumbre de $\eta_{\max}$'); ax.
117     legend(fontsize=8)
118
119     ax = axs[3]
120     hb = ax.hist2d(Qbep_s*6e4, etam_s*100, bins=70, cmap='Blues'
121 , density=True)
122     ax.plot(Qbep_mc, etam_mc, 'r+', ms=12, mew=2)
123     # elipse de covarianza 95%
124     cov = np.cov(Qbep_s*6e4, etam_s*100)
125     vals, vecs = np.linalg.eigh(cov)
126     th = np.linspace(0, 2*np.pi, 100)
127     ell = (vecs @ (np.sqrt(vals*stats.chi2.ppf(0.95,2))[:,None]*
128 np.array([np.cos(th),np.sin(th)])))
129     ax.plot(Qbep_mc+ell[0], etam_mc+ell[1], 'r-', lw=1.5, label=
130 'elipse 95%')
131     ax.set_xlabel('$Q_{BEP}$ [L/min]'); ax.set_ylabel(r'$\eta_{\max}$ [%]')
132     ax.set_title(f'(d) Conjunta (corr={rho_corr:+.2f})'); ax.
133     legend(fontsize=8)
134
135     fig.suptitle('Bomba centrifuga: incertidumbre de la
136 eficiencia y del punto de mejor eficiencia (BEP)', y=1.03)
137     plt.savefig('Figuras/fig_s6_ej4_bomba_bep.png', dpi=300,
138 bbox_inches='tight')
139     # plt.show()
140     print("Figura guardada.")

```

Listing 40: Semana 6 – Código 4: Optimización de bomba centrífuga: modelo multiplicativo y BEP. Ejecutar en Google Colab.

6.14.5. Código 5 — Velocimetría por flujo óptico desde UAV: incertidumbre correlacionada

Cadena fotogramétrica, descomposición sistemática/aleatoria y demostración de que la sistemática no se promedia sobre la flota.

```

1     """
2     Semana 6 - Ejemplo 5: Velocimetria vehicular desde UAV por
3     flujo optico.
4     Cadena fotogrametrica:
5     GSD = p_size * A / f                (m/px)

```

```

5     v     = dp * GSD / dt = dp * p_size * A / (f * dt)
6     Punto clave: A, f, p_size son COMUNES a todos los vehiculos
    de un cuadro
7     (sistematicos -> correlacion total); dp y dt son por-
    vehiculo (aleatorios).
8     La incertidumbre de la velocidad MEDIA de la flota NO
    promedia el termino
9     sistematico. Se compara el tratamiento correlacionado vs el
    ingenuo.
10    """
11    import numpy as np
12    import matplotlib
13    matplotlib.use('Agg')
14    import matplotlib.pyplot as plt
15    import matplotlib as mpl
16    from scipy import stats
17
18    mpl.rcParams.update({
19        'font.size': 10, 'axes.titlesize': 11, 'axes.labelsize':
    10,
20        'figure.dpi': 110, 'savefig.dpi': 300, 'axes.grid': True
    ,
21        'grid.alpha': 0.3, 'mathtext.fontset': 'cm',
22    })
23    rng = np.random.default_rng(505505)
24    import os; os.makedirs('Figuras', exist_ok=True) # carpeta
    de salida en Colab
25
26    # --- parametros del sensor / vuelo (comunes al cuadro =
    sistematicos) ---
27    p_size0, u_psize = 3.45e-6, 0.0                # m/px (dato de
    hoja, u~0)
28    f0, u_f = 8.8e-3, 0.05e-3                    # m, calibracion
    de camara
29    A0, u_A = 80.0, 2.5                          # m, altura AGL (
    GPS/baro) -> sistematico
30    dt0, u_dt = 1/30.0, 0.5e-3                    # s, intervalo
    entre cuadros (timestamp)
31
32    GSD0 = p_size0*A0/f0
33    print("=== EJEMPLO 5: UAV FLUJO OPTICO ===")
34    print(f"GSD nominal = {GSD0*100:.3f} cm/px")

```



```

35
36     # --- flota: N vehiculos, desplazamiento dp por vehiculo (
Tipo A, flujo) ---
37     N = 8
38     dp_true = np.array([14.0, 22.5, 9.2, 31.0, 18.7, 25.3, 12.4,
20.1]) # px/cuadro
39     u_dp = 0.4 # px,
incertidumbre del flujo optico
40     v_nom = dp_true*GSD0/dt0
41     print("v nominal por vehiculo [km/h]:", np.round(v_nom*3.6,
2))
42
43     # --- GUM por vehiculo (modelo producto): rel^2 = sum rel_i
^2 ---
44     rel_sys2 = (u_psize/p_size0)**2 + (u_f/f0)**2 + (u_A/A0)**2
+ (u_dt/dt0)**2
45     u_v_pt = v_nom*np.sqrt(rel_sys2 + (u_dp/dp_true)**2)
46     print(f"componente sistematica (rel) = {np.sqrt(rel_sys2)
*100:.3f}%")
47     print("U(v) por vehiculo [km/h] (k=2):", np.round(2*u_v_pt
*3.6, 2))
48
49     # --- velocidad media de la flota: correlaciones IMPORTANTAN
---
50     vbar = v_nom.mean()
51     # (1) tratamiento correlacionado (correcto): el termino
sistematico es comun
52     u_vbar_corr2 = (rel_sys2)*vbar**2 + (1/N**2)*np.sum((v_nom*
u_dp/dp_true)**2)
53     u_vbar_corr = np.sqrt(u_vbar_corr2)
54     # (2) tratamiento ingenuo (todo aleatorio e independiente):
promedia todo
55     u_vbar_naive = np.sqrt(np.sum(u_v_pt**2))/N
56     print(f"v_media = {vbar*3.6:.2f} km/h")
57     print(f"u(v_media) correlacionado = {u_vbar_corr*3.6:.3f} km
/h")
58     print(f"u(v_media) ingenuo = {u_vbar_naive*3.6:.3f}
km/h")
59     print(f"subestimacion del ingenuo = factor {u_vbar_corr/
u_vbar_naive:.2f}")
60
61     # --- Monte Carlo confirmatorio ---

```

```

62     M = 200_000
63     # sistematicos: UN sorteo por prueba, COMPARTIDO por la
        flota
64     A_s = rng.normal(A0, u_A, M)
65     f_s = rng.normal(f0, u_f, M)
66     dt_s = rng.normal(dt0, u_dt, M)
67     GSD_s = p_size0*A_s/f_s
68     vbar_corr_mc = np.empty(M)
69     vbar_naive_mc = np.empty(M)
70     for j in range(N):
71         dp_s = dp_true[j] + rng.normal(0, u_dp, M)      # aleatorio
        por vehiculo
72         vbar_corr_mc += dp_s*GSD_s/dt_s
73         # ingenuo: cada vehiculo sortea TODO de forma independiente
74         A_i = rng.normal(A0, u_A, M); f_i = rng.normal(f0, u_f, M);
        dt_i = rng.normal(dt0, u_dt, M)
75         vbar_naive_mc += (dp_true[j]+rng.normal(0,u_dp,M))*(p_size0*
        A_i/f_i)/dt_i
76         vbar_corr_mc /= N; vbar_naive_mc /= N
77         print(f"[MC] u(v_media) correlacionado = {vbar_corr_mc.std(
        ddof=1)*3.6:.3f} km/h")
78         print(f"[MC] u(v_media) ingenuo          = {vbar_naive_mc.std(
        ddof=1)*3.6:.3f} km/h")
79
80     # --- figura ---
81     fig, axs = plt.subplots(1, 4, figsize=(16, 3.7))
82
83     ax = axs[0]
84     order = np.argsort(v_nom)
85     yp = np.arange(N)
86     ax.errorbar(v_nom[order]*3.6, yp, xerr=2*u_v_pt[order]*3.6,
        fmt='o',
87     color='#0b3d91', ms=6, capsize=3, lw=1.2)
88     ax.axvline(vbar*3.6, color='g', ls='--', lw=1.2, label='
        media de la flota')
89     ax.axvspan((vbar-2*u_vbar_corr)*3.6, (vbar+2*u_vbar_corr)
        *3.6, color='#d9ead3',
90     alpha=0.7, label='$U$(media) correl.')
91     ax.set_yticks(yp); ax.set_yticklabels([f'V{ i+1}' for i in
        order])
92     ax.set_xlabel('velocidad [km/h]'); ax.set_title('(a)
        Velocidades $\backslash$pm U$ (k=2)')

```

```

93     ax.legend(fontsize=7.5, loc='lower right')
94
95     ax = axs[1]
96     labels = ['$A$ (altura)', '$f$ (focal)', '$\\Delta t$', '$\\Delta p$ (flujo)']
97     # para un vehiculo tipico (mediano)
98     jt = np.argsort(dp_true)[N//2]
99     comps = np.array([(u_A/A0)**2, (u_f/f0)**2, (u_dt/dt0)**2, (u_dp/dp_true[jt])**2])
100     pc = 100*comps/comps.sum()
101     ax.bar(labels, pc, color=['#b22222', '#b22222', '#b22222', '#0b3d91'])
102     for i, p in enumerate(pc):
103         ax.text(i, p+1, f'{p:.0f}%', ha='center', fontsize=8)
104         ax.set_ylabel('% de la varianza de $v$')
105         ax.set_title('(b) Presupuesto (rojo=sistemico)'); ax.set_ylim(0, max(pc)*1.2)
106
107     ax = axs[2]
108     ax.hist(vbar_naive_mc*3.6, bins=120, density=True, color='#f4cccc', edgecolor='none',
109            label=f'ingenuo (u={vbar_naive_mc.std(ddof=1)*3.6:.2f})')
110     ax.hist(vbar_corr_mc*3.6, bins=120, density=True, color='#9fc5e8', edgecolor='none',
111            alpha=0.8, label=f'correlacionado (u={vbar_corr_mc.std(ddof=1)*3.6:.2f})')
112     ax.set_xlabel('velocidad media de la flota [km/h]'); ax.set_ylabel('densidad')
113     ax.set_title('(c) Efecto de la correlacion'); ax.legend(fontsize=7.5)
114
115     ax = axs[3]
116     Ns = np.arange(1, 25)
117     u_corr = np.sqrt(rel_sys2*vbar**2 + (1/Ns**2)*N*(vbar*u_dp/dp_true.mean())**2)*3.6
118     u_naive = np.sqrt((rel_sys2 + (u_dp/dp_true.mean())**2)*vbar**2/Ns)*3.6
119     ax.plot(Ns, u_corr, '-o', color='#0b3d91', ms=3, lw=1.4, label='correlacionado')
120     ax.plot(Ns, u_naive, '-s', color='#b22222', ms=3, lw=1.4, label='ingenuo')
121     ax.axhline(np.sqrt(rel_sys2)*vbar*3.6, color='gray', ls=':',

```

```

122     lw=1.2, label='piso sistematico')
123     ax.set_xlabel('numero de vehiculos $N$'); ax.set_ylabel('$u$
(media) [km/h]')
124     ax.set_title('(d) La sistematica no promedia'); ax.legend(
fontsize=8)
125
126     fig.suptitle('UAV + flujo optico: incertidumbre de la
velocidad y el papel de las correlaciones (sistematico vs
aleatorio)', y=1.03)
127     plt.savefig('Figuras/fig_s6_ej5_uav_flujo.png', dpi=300,
bbox_inches='tight')
128     # plt.show()
print("Figura guardada.")

```

Listing 41: Semana 6 – Código 5: Velocimetría por flujo óptico desde UAV: incertidumbre correlacionada. Ejecutar en Google Colab.

6.14.6. Código 6 — Exactitud de un sistema RAG: soporte acotado

Cascada recuperación–generación, propagación con distribuciones Beta y comparación con intervalos normal y de Wilson.

```

1      """
2      Semana 6 - Ejemplo 6: Incertidumbre de la exactitud de un
sistema RAG
3      industrial estimada con un conjunto de evaluacion FINITO.
4      Modelo de composicion: a_e2e = a_ret * a_gen/ret (dos
proporciones)
5      Cada proporcion se estima de n ensayos -> incertidumbre Tipo
A binomial.
6      Punto clave: con n pequeno y p cerca de 1, el intervalo GUM-
normal puede
7      exceder [0,1] (no fisico); MC/Beta respeta el soporte. Se
comparan
8      GUM (LPU), Monte Carlo (posterior Beta) y Wilson.
9      """
10     import numpy as np
11     import matplotlib
12     matplotlib.use('Agg')
13     import matplotlib.pyplot as plt
14     import matplotlib as mpl
15     from scipy import stats
16

```

```

17     mpl.rcParams.update({
18         'font.size': 10, 'axes.titlesize': 11, 'axes.labelsize':
19         10,
20         'figure.dpi': 110, 'savefig.dpi': 300, 'axes.grid': True
21     },
22     'grid.alpha': 0.3, 'mathtext.fontset': 'cm',
23     })
24     rng = np.random.default_rng(606606)
25     import os; os.makedirs('Figuras', exist_ok=True) # carpeta
26     de salida en Colab
27
28     # --- resultados de la evaluacion (conteos) ---
29     n_ret, k_ret = 60, 54 # 54/60 recuperaciones
30     correctas
31     n_gen, k_gen = 48, 46 # 46/48 generaciones correctas
32     (alta exactitud, n pequeno)
33     p_ret = k_ret/n_ret
34     p_gen = k_gen/n_gen
35     a0 = p_ret*p_gen
36     print("=== EJEMPLO 6: SISTEMA RAG ===")
37     print(f"a_ret={p_ret:.4f} (n={n_ret}), a_gen={p_gen:.4f} (n
38     ={n_gen}), a_e2e={a0:.4f}")
39
40     # --- Tipo A binomial (error estandar de cada proporcion)
41     ---
42     u_ret = np.sqrt(p_ret*(1-p_ret)/n_ret)
43     u_gen = np.sqrt(p_gen*(1-p_gen)/n_gen)
44     # --- GUM (LPU) modelo producto ---
45     u_a = a0*np.sqrt((u_ret/p_ret)**2 + (u_gen/p_gen)**2)
46     U_a = 1.96*u_a
47     lo_gum, hi_gum = a0-U_a, a0+U_a
48     print(f"u_ret={u_ret:.4f}, u_gen={u_gen:.4f}, u(a_e2e)={u_a
49     :.4f}")
50     hi_gen_normal = p_gen + 1.96*u_gen
51     print(f"[GUM] componente a_gen normal 95% sup = {
52     hi_gen_normal:.4f} (excede 1? {hi_gen_normal>1})")
53     print(f"[GUM] 95%: [{lo_gum:.4f}, {hi_gum:.4f}] (a_e2e
54     excede 1? {hi_gum>1})")
55
56     # --- Monte Carlo con posterior Beta (Jeffreys: a=k+0.5, b=n
57     -k+0.5) ---
58     M = 300_000
    
```

```

48     ret_s = rng.beta(k_ret+0.5, n_ret-k_ret+0.5, M)
49     gen_s = rng.beta(k_gen+0.5, n_gen-k_gen+0.5, M)
50     a_s = ret_s*gen_s
51     a_mc = a_s.mean(); u_mc = a_s.std(ddof=1)
52     lo_mc, hi_mc = np.percentile(a_s, [2.5, 97.5])
53     print(f"[MC ] media={a_mc:.4f}, u={u_mc:.4f}, 95%: [{lo_mc:.4f}, {hi_mc:.4f}]")
54
55     # --- Wilson para a_e2e via delta-method sobre log (
referencia) ---
56     def wilson(k, n, z=1.96):
57         p = k/n
58         den = 1+z**2/n
59         c = (p + z**2/(2*n))/den
60         hw = z*np.sqrt(p*(1-p)/n + z**2/(4*n**2))/den
61         return c-hw, c+hw
62     lo_rw, hi_rw = wilson(k_ret, n_ret); lo_gw, hi_gw = wilson(
k_gen, n_gen)
63     # producto de intervalos (conservador)
64     lo_w, hi_w = lo_rw*lo_gw, hi_rw*hi_gw
65     print(f"[Wilson prod] 95%: [{lo_w:.4f}, {hi_w:.4f}]")
66
67     # --- proyeccion: respuestas correctas esperadas por 1000
consultas ---
68     print(f"Por 1000 consultas: {a0*1000:.0f} correctas; 95% MC:
[{lo_mc*1000:.0f}, {hi_mc*1000:.0f}]")
69
70     # --- u(a_e2e) en funcion del tamaño del conjunto de
evaluacion ---
71     ns = np.array([20, 30, 50, 80, 120, 200, 400, 800])
72     u_vs_n = a0*np.sqrt((np.sqrt(p_ret*(1-p_ret)/ns)/p_ret)**2
73 + (np.sqrt(p_gen*(1-p_gen)/ns)/p_gen)**2)
74
75     # --- figura ---
76     fig, axs = plt.subplots(1, 4, figsize=(16, 3.7))
77
78     ax = axs[0]
79     xx = np.linspace(0.6, 1.08, 500)
80     ax.plot(xx, stats.beta.pdf(xx, k_ret+0.5, n_ret-k_ret+0.5),
color='#0b3d91', lw=1.8,
81     label=f'$a_{\{ret\}}$ Beta ({k_ret}/{n_ret})')
82     ax.plot(xx, stats.beta.pdf(xx, k_gen+0.5, n_gen-k_gen+0.5),

```

```

color='#b22222', lw=1.8,
83     label=f'$a_{\{gen\}}$ Beta ( $\{k_{gen}\}/\{n_{gen}\}$ )')
84     # aproximacion normal del componente alto: cola no fisica >
1
85     ng = stats.norm.pdf(xx, p_gen, u_gen)
86     ax.plot(xx, ng, color='#b22222', lw=1.2, ls='--', label='$a_{\{gen\}}$ normal (GUM)')
87     ax.fill_between(xx[xx>1], 0, ng[xx>1], color='red', alpha
=0.3)
88     ax.axvline(1.0, color='k', lw=1.2)
89     ax.text(1.002, ax.get_ylim()[1]*0.5, 'cola\n $>1$ ', fontsize
=7.5, color='red')
90     ax.set_xlabel('proporcion'); ax.set_ylabel('densidad
posterior')
91     ax.set_title('(a) La normal asigna masa a  $p>1$ '); ax.legend
(fontsize=7.2, loc='upper left')
92
93     ax = axs[1]
94     ax.hist(a_s, bins=120, density=True, color='#9ecae1',
edgecolor='none', label='MC (Beta)')
95     xg = np.linspace(a0-4*u_a, min(a0+4*u_a, 1.05), 300)
96     ax.plot(xg, stats.norm.pdf(xg, a0, u_a), 'r-', lw=1.6, label
='GUM (normal)')
97     ax.axvline(a0, color='#b22222', ls=':', lw=1.2)
98     ax.axvline(a_mc, color='k', ls=':', lw=1.2)
99     ax.set_xlabel('$a_{e2e}$'); ax.set_ylabel('densidad')
100    ax.set_title(f'(b) Asimetria por soporte ( $\gamma_1=\{stats
.skew(a_s):+.2f\}$ )'); ax.legend(fontsize=8, loc='upper left')
101
102    ax = axs[2]
103    methods = ['GUM\n(normal)', 'Monte Carlo\n(Beta)', 'Wilson\n
(producto)']
104    los = [lo_gum, lo_mc, lo_w]; his = [hi_gum, hi_mc, hi_w]
105    cen = [a0, a_mc, a0]
106    for i,(l,h,c) in enumerate(zip(los,his,cen)):
107    ax.plot([l,h],[i,i],'-', color='#0b3d91', lw=3)
108    ax.plot(c,i,'o',color='#b22222',ms=7)
109    ax.axvline(1.0, color='k', ls='--', lw=1.2)
110    ax.set_yticks(range(3)); ax.set_yticklabels(methods,
fontsize=8.5)
111    ax.set_xlabel('$a_{e2e}$ (intervalo 95%)')
112    ax.set_title('(c) Intervalos de cobertura'); ax.set_ylim

```

```

113     (-0.6, 2.6)
114     ax = axs[3]
115     ax.loglog(ns, u_vs_n, '-o', color='#0b3d91', ms=4, lw=1.5,
116     label='$u(a_{e2e})$')
117     ax.loglog(ns, u_a*np.sqrt(n_ret/ns), 'r--', lw=1.2, label=r'
118     $\propto n^{-1/2}$')
119     for utgt, lbl in [(0.01, '1%')]:
120     ax.axhline(utgt, color='gray', ls=':', lw=1)
121     ax.text(ns[0], utgt*1.1, f'objetivo {lbl}', fontsize=7.5,
122     color='gray')
123     ax.set_xlabel('tamano del conjunto de evaluacion $n$'); ax.
124     set_ylabel('$u(a_{e2e})$')
125     ax.set_title('(d) Precision vs tamano de evaluacion'); ax.
126     legend(fontsize=8)

127     fig.suptitle('Sistema RAG: incertidumbre de la exactitud por
128     muestra finita (GUM, Monte Carlo y Wilson)', y=1.03)
129     plt.savefig('Figuras/fig_s6_ej6_rag.png', dpi=300,
130     bbox_inches='tight')
131     # plt.show()
132     print("Figura guardada.")

```

Listing 42: Semana 6 – Código 6: Exactitud de un sistema RAG: soporte acotado. Ejecutar en Google Colab.

6.15. Ejercicios Reto — Tarea Semana 6

Los Ejercicios 1 y 2 son obligatorios. El Ejercicio 3 otorga punto extra. Todos deben anclarse al *proyecto doctoral propio* del estudiante: el mensurando, las fuentes de incertidumbre y los datos provienen de su propia investigación.

6.15.1. Ejercicio 1 (obligatorio) — Modelo de medición y propagación analítica

Formule el modelo de medición $Y = f(X_1, \dots, X_n)$ de una magnitud central de su proyecto. Entregue: (i) la ecuación del modelo y la identificación de cada X_i como Tipo A o Tipo B, justificando la distribución asignada por máxima entropía; (ii) la tabla de presupuesto de incertidumbre con los coeficientes de sensibilidad $c_i = \frac{\partial f}{\partial X_i}$ evaluados numéricamente; (iii) la incertidumbre combinada $u_c(y)$ por LPU (178), *incluyendo* al menos un término de correlación si su modelo lo requiere; y (iv) la incertidumbre expandida $U = k u_c$ con k obtenido por Welch–Satterthwaite (180). Reporte el resultado en la forma $y \pm U$ con su nivel de cobertura.

Rúbrica.

- 3 Modelo correcto, clasificación A/B justificada, presupuesto completo con sensibilidades numéricas, u_c con correlación y U con k por Welch–Satterthwaite. Reporte metrológicamente correcto.
- 2 Modelo y u_c correctos pero sin término de correlación cuando corresponde, o k asintótico (1.96) sin justificar los grados de libertad.
- 1 Propagación incompleta: faltan coeficientes de sensibilidad o el presupuesto no cierra.
- 0 Sin modelo de medición o sin propagación analítica.

6.15.2. Ejercicio 2 (obligatorio) — Propagación Monte Carlo y validación de la GUM

Implemente en Google Colab la propagación Monte Carlo del mismo modelo del Ejercicio 1, con $M \geq 10^5$ pruebas y generador reproducible (`numpy.random.default_rng(semilla)`). Entregue: (i) el histograma del mensurando con el intervalo de cobertura del 95 % por percentiles; (ii) la comparación numérica de los intervalos GUM y Monte Carlo; (iii) la aplicación del criterio de validación (182) con una tolerancia δ explícita derivada del número de cifras significativas que desea reportar; y (iv) una decisión razonada: ¿reporta el resultado GUM o el Monte Carlo, y por qué? Si su modelo es no lineal, asimétrico o de soporte acotado, discuta el mecanismo de la discrepancia.

Rúbrica.

- 3 Implementación reproducible, intervalo por percentiles correcto, validación con δ explícita y decisión justificada por el mecanismo físico/estadístico de la discrepancia.
- 2 Monte Carlo correcto pero validación sin δ explícita, o decisión no justificada.
- 1 Simulación que corre pero con generador no reproducible o sin intervalo de cobertura bien construido.
- 0 Sin implementación Monte Carlo funcional.

6.15.3. Ejercicio 3 — Procedimiento Monte Carlo adaptativo

Aplique el procedimiento adaptativo de la JCGM 101: en lugar de fijar M arbitrariamente, incremente M por bloques hasta que los límites del intervalo de cobertura sean numéricamente estables a la última cifra significativa deseada (tolerancia δ). Entregue la curva de convergencia de los límites del intervalo y de u_c frente a M , verificando empíricamente la tasa $O(M^{-1/2})$ de (181), y reporte el M final adoptado con su justificación.

Rúbrica.